TrustTrail: Accountable, Purpose-Bound Attribute Release for Privacy-Aware Federated Identity Management

Vijender Yadav (23BCS132), Ansh Hardaha (23BCS016), and Sourabh (23BEC055)

Abstract

In a typical campus single sign-on deployment every service receives the same identifier and a fixed set of personal claims, no release is tied to a stated purpose, and students cannot see who received what. The survey by Werner, Westphall and Westphall shows that accountability and use limitation are among the least covered privacy characteristics in identity management. We present TrustTrail, a disclosure layer beside an unmodified OpenID Connect provider (Keycloak 26). Keycloak caps which of four disclosure profiles a service may obtain and issues per-service pairwise pseudonyms; a custom mapper removes the shared session identifier. TrustTrail is the only source of attribute values. It releases them only under a purpose declaration that the service has signed, which names the attributes, the purpose and a deletion date, and it returns a signed receipt. Releases and refused rule breaks are recorded in a hash-chained ledger whose Merkle roots are anchored in an Ethereum contract, and violations lower the service's tier automatically. On one machine, the layer added a median of 3.2 ms per login (95% CI 3.0–3.4 ms). Conformance tests detected six tamper attacks and 45 scripted violations as designed, with no false positives in 90 honest releases. On the Sepolia test network, anchoring cost 254,408 gas per batch whatever its size. TrustTrail provides evidence of what services promised and received. It does not prove deletion, and its protection against the operator covers only history that has already been anchored.

Index Terms

Federated identity management, privacy, OpenID Connect, pairwise pseudonyms, purpose limitation, accountability, tamper-evident logging, Merkle tree, blockchain anchoring, trust management.

I. Introduction

A UNIVERSITY runs many online services for its students: a digital library, a learning management system (LMS), an examination portal, and often third-party applications as well. Identity federation and single sign-on (SSO) made this manageable: the student logs in once at an identity provider (IdP), and each service, called a relying party, accepts the IdP's assertion instead of keeping its own password database [1], [2]. Protocols such as SAML 2.0 [3] and OpenID Connect (OIDC) [4] on top of OAuth 2.0 [5] are mature, and open-source IdPs such as Keycloak [6] make SSO cheap to deploy.

SSO solves the login problem. It does little for privacy, and in some ways it makes privacy worse. The project started from a simple question: what does a typical campus SSO configuration actually send to each service? The answer shows three problems.

Over-release. The easiest configuration gives every client the same set of claims: name, e-mail address, roll number, department and year. The library needs to know that the borrower is a current student. It does not need the student's name, and it certainly does not need the roll number. Studies of permission systems show that applications routinely ask for more than they need when nothing stops them [7], [8].

Linkability. If the IdP gives every service the same subject identifier, or the same e-mail address, any two services can join their records. Even when identifiers differ, a session identifier shared across clients in one SSO session can link them, as we show in Section VII. Unlinkability is one of the three privacy properties that Birrell and Schneider identify for federated identity [1], and it is the one that a default deployment gives up first.

No record and no recourse. After a release, the student cannot find out which service received which attribute, for what purpose, or until when. If a service later asks for something it never said it needed, nobody notices, and the

The authors are B.Tech. students (7th semester) at the Indian Institute of Information Technology Dharwad, Karnataka, India.

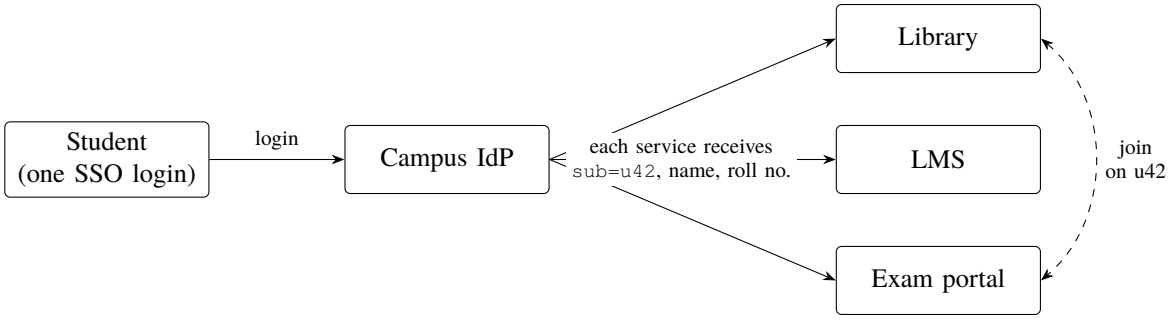


Fig. 1. The problem in a default SSO deployment. Every service receives the same identifier and the same personal claims, whatever it needs, so services can join their records; the student has no record of the releases.

service keeps the same access as before. Data protection law expects the opposite. The EU General Data Protection Regulation requires that personal data be collected for specified purposes, limited to what is necessary, kept no longer than necessary, and that the controller be able to demonstrate compliance [9]. India's Digital Personal Data Protection Act, 2023 likewise ties processing to a specified purpose and requires erasure once that purpose is no longer served [10]. These are statements about what should happen. A campus IdP gives no evidence of whether it did.

Our starting point was the survey by Werner, Westphall and Westphall, "Cloud identity management: A survey on privacy strategies" [11]. The survey proposes a taxonomy of nine privacy characteristics for identity management (transparency, controllability, minimization, accountability, data quality, use limitation, user-friendliness, trust and obfuscation), maps 23 published proposals and six European research projects against it, and lists open challenges. Reading its tables closely, we found that the two characteristics that matter most for the third problem are also the least covered. Only 2 of the 23 proposals address accountability and only 5 address use limitation; the four works the authors themselves present as solutions in progress address neither [11, Tables 2–3]. The survey also names "ensure the implementation of policies" as an open challenge and observes that several surveyed proposals were never implemented or validated. Section III analyses the survey in detail.

This paper presents *TrustTrail*, a system that addresses these gaps for an OIDC federation and that we built and evaluated end to end. TrustTrail keeps the IdP unmodified in its protocol behaviour and adds a disclosure layer beside it. Its design follows one loop:

$$\textit{signed promise} \rightarrow \textit{ledger record} \rightarrow \textit{violation detected} \rightarrow \textit{trust lowered} \rightarrow \textit{less data released.}$$

A service first signs a purpose declaration that names the attributes it wants, why it wants them and when it will delete them. TrustTrail releases attribute values only under such a declaration and only up to the disclosure profile that the service's current trust tier allows. Every release, and every refused request that breaks a rule, becomes an entry in a hash-chained ledger. Batches of ledger entries are summarised as Merkle roots and anchored in an Ethereum smart contract, so that once a batch is anchored on a public chain, not even the operator can rewrite it without detection. A service that breaks a rule loses trust, and the IdP stops granting it the more revealing profiles.

A. Research questions

We framed the work around five questions.

RQ1 Can a federation give each service only the attributes of an agreed disclosure profile, and stop services from linking a student across services, while the IdP keeps speaking standard OIDC?

RQ2 Can every release be bound to a purpose and a deletion date that the service has signed, so that a service cannot later deny what it promised?

RQ3 Can the record of releases be made tamper-evident against an insider who controls the database, including the operator, without putting personal data on a public ledger?

RQ4 Can rule-breaking that the disclosure layer can observe automatically reduce what the offending service receives in future?

RQ5 What do these mechanisms cost in login latency and in blockchain fees?

B. Contributions

1) A gap analysis of Werner et al. [11] that uses the survey's own tables to show where its taxonomy is least covered, and that maps each of its five open challenges to a concrete design requirement (Section III).
2) A design that splits disclosure between the IdP and a separate release service: Keycloak decides *which* profile a service may obtain, using per-service scope assignment as a tier cap, pairwise subject identifiers and a custom protocol mapper that removes the shared session identifier; TrustTrail decides *what* the profile contains and is the only path for attribute values (Sections VI and VII).
3) Signed purpose declarations and release receipts based on JSON Web Signatures [12], which bind every release to the attributes, purpose and deletion date that the service promised, and give the student portable, verifiable evidence of each release.
4) An accountability chain made of a hash-chained ledger, per-release receipts and Merkle-root anchoring in an Ethereum contract that only accepts contiguous batches, with a verification procedure that a student can run using only a downloaded receipt and the chain.
5) A dynamic trust model in which three observable violation rules lower a service's score and its tier, and the tier is enforced both at the IdP (login is refused) and at the release service (already issued tokens get the lower cap).
6) An open implementation (Keycloak 26.8.1, Python 3.12/FastAPI, Solidity 0.8.28, Hardhat 3) with 151 Python tests and 6 contract tests, deployed both on a local chain and on the Sepolia test network, and an evaluation of latency, tamper detection, violation detection and anchoring cost (Sections VIII and IX).

What is new and what is not. Each building block of TrustTrail is established: pairwise identifiers come from OIDC [4], signatures from JOSE [12], hash chains from secure audit logging [13], Merkle trees and domain-separated hashing from Certificate Transparency [14], and anchoring from timestamping services [15]. We do not claim any new cryptography. The contribution is the way these parts are combined around a standard IdP to address the gaps that the base survey leaves open. Four elements are, to our knowledge, not found together in the surveyed works: purpose declarations signed by the *service*, not policies stated by the user; a ledger that records refused rule breaks as well as releases; a contract that accepts only contiguous batches, so a truncated ledger is visible; and a trust tier that observed violations lower, enforced at the IdP's scope assignment. We also report a practical finding that we did not expect: in Keycloak 26, pairwise identifiers alone do not prevent linking, because of a shared session identifier, and a custom mapper is needed to remove it.

We are careful about what TrustTrail does not do. It records what a service promised and what it received, and it refuses releases that break the promise. It cannot see what a service does with data afterwards, so it cannot prove deletion. The IdP still sees every login. Section X discusses these and other limits.

C. Organisation

Section II introduces the protocols and primitives that TrustTrail builds on. Section III analyses the base survey and derives the gaps we address, and Section IV reviews related work. Section V states the threat model and design goals. Sections VI and VII present the architecture and the detailed design, and Section VIII the implementation. Section IX reports the evaluation. Sections X, XI and XII discuss limitations, future work and conclusions.

II. Background

This section introduces the protocols, cryptographic building blocks and privacy principles that the rest of the paper relies on. Readers familiar with OIDC and Merkle trees can skip to Section III.

A. Identity management, federation and single sign-on

Identity management (IdM) covers how identities are created, authenticated, described by attributes and eventually removed. In a federation, several organisations agree to trust each other's identity assertions. One party, the identity provider (IdP), authenticates the user and issues an assertion; the service provider (SP), or relying party (RP) in OIDC terms, accepts the assertion and makes its own authorisation decision [1], [11]. Single sign-on (SSO) follows: once the user has a session at the IdP, further services can be reached without logging in again.

Federations differ in how much the members trust each other. Ferdous and Poet distinguish *fully trusted*, *semi-trusted* and *untrusted* entities in dynamic SAML federations, depending on whether a legal agreement backs the relationship [2]. TrustTrail reuses these three names for its service tiers and adds a fourth, *blocked*, for services that have lost trust through violations.

From a privacy point of view, the IdP is in a powerful position. It knows every user's attributes and sees every login to every service. Birrell and Schneider characterise federated IdM systems by three properties that users may want [1]: *undetectability* (others cannot tell that a transaction happened), *unlinkability* (transactions cannot be connected to each other or to the user) and *confidentiality* (the user controls who learns their attributes). Pfitzmann and Hansen give the standard terminology for anonymity, unlinkability and pseudonymity [16]. A *pseudonym* is an identifier other than the user's real name; it is *relationship-specific* when the user has a different pseudonym for each communication partner, which prevents partners from linking the user by identifier.

B. OAuth 2.0 and OpenID Connect

OAuth 2.0 [5] lets a client obtain an *access token* from an authorisation server in order to call a protected API. The client asks for *scopes*, which name the permissions it wants, and the server decides which scopes to grant. Two OAuth flows matter here. In the *authorization code* flow, the user's browser is redirected to the server, the user authenticates, and the client exchanges a one-time code for tokens over a back channel. In the *client credentials* flow, a client authenticates as itself, with no user involved; TrustTrail uses it to authenticate services that register their keys.

OpenID Connect [4] adds authentication on top of OAuth. The server, now an OpenID Provider, issues an *ID token*: a signed JSON Web Token (JWT) [17] whose claims state who the user is (`sub`), who issued the token (`iss`), for which client it is meant (`aud`) and when it was issued and expires. Additional user attributes, such as name or e-mail address, can be placed in the ID token, the access token, or returned from a userinfo endpoint, depending on the requested scopes and the provider's configuration. Keycloak [6] controls this per client through *client scopes* and *protocol mappers*. A client scope bundles claims; it can be assigned to a client as a *default* scope (always granted) or an *optional* scope (granted only when requested). A protocol mapper is a plug-in that adds, changes or removes claims in the tokens that a client receives.

1) Pairwise subject identifiers: OIDC defines two subject identifier types [4, Sec. 8]. With the *public* type, every client receives the same `sub` for a user. With the *pairwise* type, each client, or more exactly each *sector* of clients, receives a different `sub`, so clients cannot correlate the user by identifier. The specification suggests computing the pairwise value as a hash of the sector identifier, the local account identifier and a secret salt. Keycloak's SHA-256 pairwise mapper does this and then formats the 32-byte digest as a name-based (version 3) UUID; the sector is the host name of the client's redirect URIs unless a sector identifier URI is configured. The salt is configured per client mapper. In the OIDC model, clients that share a sector are meant to receive the same value; when every client has its own random salt, as in our deployment, even clients in one sector receive different values. Whoever can read the salts can recompute every pseudonym, so the salts are the key to linking and must stay confidential. Pairwise identifiers are also recommended by NIST's federation guidelines [18].

2) Session identifiers: Keycloak adds a session identifier claim, `sid`, to the tokens it issues. All clients that a user reaches within one SSO session receive the same `sid` value. The authorisation response can also carry a `session_state` parameter derived from the session. As Section VII shows, these values defeat pairwise identifiers: two services that compare `sid` values can link the two pseudonyms of the same student.

C. JSON Web Signatures and keys

A JSON Web Signature (JWS) [12] protects a payload with a digital signature or MAC. Its *compact serialisation* is three base64url strings separated by dots: a protected header, the payload and the signature. The header names the algorithm (`alg`), may name the signing key (`kid`) and may state a media type (`typ`) so that a JWS meant for one purpose is not accepted for another. TrustTrail uses two algorithms from the JSON Web Algorithms specification [19]: ES256 (ECDSA on curve P-256 with SHA-256) and RS256 (RSASSA-PKCS1-v1_5 with SHA-256). Public keys are exchanged as JSON Web Keys (JWK) [20]. The JWK thumbprint [21] is the SHA-256 hash of a canonical JSON object made of the key's required members in lexicographic order; it gives a key a stable identifier that is derived from the key itself, which TrustTrail uses as the `kid`.

A signed JWS gives *non-repudiation* in a practical sense: if the verifier keeps the JWS, it can later show a third party that the holder of the private key signed exactly this payload. TrustTrail relies on this property for purpose declarations, which services sign, and for release receipts, which TrustTrail signs.

D. Hash chains and tamper-evident logs

A cryptographic hash function such as SHA-256 maps any input to a fixed-size digest; it is infeasible to find two inputs with the same digest or to change an input without changing its digest. A *hash chain* links records by including the digest of the previous record in each new record. Haber and Stornetta used this idea to timestamp digital documents [22], and Schneier and Kelsey applied it to audit logs, so that an attacker who takes over a logging machine cannot undetectably change entries written before the takeover [13]. A chain alone has a known weakness: whoever controls the whole log can rewrite an entry and recompute every later hash, or cut off the newest entries. Detecting that requires some value that the attacker cannot change, such as a hash that has been published elsewhere or handed to a third party [23], [24].

E. Merkle trees

A Merkle tree [25] hashes a list of items in pairs, then hashes the results in pairs, until one *root* hash remains. The root commits to every item and its position. To prove that an item is in the tree, it is enough to give the hashes of its siblings on the path to the root, which takes $\lceil \log_2 n \rceil$ hashes for n items; the verifier recomputes the root and compares. Certificate Transparency (CT) uses Merkle trees for public, append-only logs of TLS certificates [26]. Its specification, RFC 6962 [14] (updated by RFC 9162 [27]), prefixes leaf inputs with the byte `0x00` and interior-node inputs with `0x01`. This domain separation stops an attacker from presenting an interior node as a leaf. TrustTrail follows these conventions.

F. Blockchains, smart contracts and anchoring

A blockchain is a replicated ledger in which blocks of transactions are linked by hashes and agreed on by many independent nodes, so that a confirmed transaction is very hard to remove or change [28]. Ethereum [29] adds *smart contracts*: programs stored on the chain whose functions are called by transactions. A contract can keep state, emit *events* that clients can search, and reject calls that break its rules. Every operation costs *gas*, paid for in the chain's currency; writing to contract storage is among the most expensive operations. Contracts are usually written in Solidity [30] and tested on a local development chain such as Hardhat [31] before deployment to a public test network such as Sepolia or to the main network.

Anchoring uses a blockchain as the external, hard-to-change place where a log's hash is published. Instead of writing each record, a system periodically writes one Merkle root that covers many records. OpenTimestamps [15] and Chainpoint [32] anchor document hashes this way. Anchoring keeps the data off chain, which matters for personal data that must not become public and permanent, and keeps the cost per record low.

G. Privacy principles in law and design

The GDPR [9] lists principles for processing personal data in Article 5. Four of them shape TrustTrail's goals: *purpose limitation* (data is collected for specified, explicit purposes), *data minimisation* (adequate, relevant and limited to what is necessary), *storage limitation* (kept no longer than necessary) and *accountability* (the controller is responsible for, and able to demonstrate, compliance). Pseudonymised data, such as a ledger that records per-service pseudonyms, remains personal data under the GDPR as long as it can be attributed to a person with additional information (Recital 26).

India's Digital Personal Data Protection Act, 2023 (DPDP Act) [10] applies to student data held by Indian institutes. In its terms, the institute is the *Data Fiduciary*, which decides the purpose and means of processing, the student is the *Data Principal*, and a service that processes data on the institute's behalf is a *Data Processor* (Section 2). Processing needs either consent given after a notice of the data and purpose (Sections 5 and 6) or one of the "certain legitimate uses" listed in Section 7. A Data Fiduciary must erase personal data when consent is withdrawn or as soon as it is reasonable to assume that the specified purpose is no longer being served, unless a law requires retention, and must make its processors erase it too (Section 8(7)); it must also establish an effective

TABLE I
TERMS USED IN THIS PAPER

Term	Meaning in TrustTrail
IdP	Keycloak, the campus OpenID Provider. Authenticates students and issues tokens.
Service	A relying party, such as the library, the LMS or the exam portal.
Profile	One of four disclosure levels: *anonymous*, *pseudonym*, *partial*, *total*. Each maps to a fixed attribute set.
Tier	A service's trust level: *fully_trusted*, *semi_trusted*, *untrusted* or *blocked*. Each tier has a profile cap.
Base tier	The tier in the service's agreement with the institute; its cap is the most the service may ever ask for.
Pairwise `sub`	The per-service pseudonym of a student.
Purpose declaration	A JWS signed by a service: attributes, purpose, deletion date (`retention_until`).
Release	One delivery of attribute values from TrustTrail to a service.
Receipt	A JWS signed by TrustTrail that describes one release.
Ledger	The append-only, hash-chained table of releases, violations and trust resets.
Violation	A refused request that breaks one of three rules; recorded in the ledger and counted against trust.
Anchor	A Merkle root of a contiguous batch of ledger entries, stored in the `TrustTrailAnchor` contract.

mechanism to redress the grievances of Data Principals (Section 8(10)). A Data Principal has rights to a summary of the data processed and of who received it (Section 11), to correction and erasure (Section 12), and to grievance redressal (Section 13). Processing the data of a child, meaning anyone under eighteen, requires verifiable consent of a parent or lawful guardian (Sections 2(f) and 9). Several of these duties concern records that TrustTrail creates, and Section X returns to them.

Cavoukian's *Privacy by Design* principles [33] ask that privacy be the default setting, embedded into the design, and visible and transparent to the user. The Kantara Initiative's Consent Receipt specification [34] proposes giving users a record of each consent they grant, an idea close to TrustTrail's release receipts.

Table I summarises the terms used in the rest of the paper.

III. The Base Paper and Its Gaps

TrustTrail is a follow-up to the survey by Werner, Westphall and Westphall [11], published in *Computer Networks* in 2017. This section summarises what the survey contributes, reads its tables quantitatively, and derives the gaps that TrustTrail addresses. All counts below come from the survey's own tables; we did not re-score the surveyed works.

A. What the survey does

The survey starts from the observation that, even with IdM systems, "leakage of user data, the distribution of unnecessary attributes, and a lack of control on the dissemination of personal data are recurrent in cloud environments" [11]. It asks three research questions: which IdM model is needed to ensure privacy in cloud computing, which techniques best address the lack of privacy control in interactions, and how federation partners should agree on privacy for authentication and authorisation. To answer them, the authors ran a systematic mapping. A keyword search found 198 papers, which were filtered by title and abstract and then by content.

The central contribution is a taxonomy of privacy characteristics for IdM in the cloud, shown as a mind map in the survey's Fig. 3. It has nine characteristics, each refined into two mechanisms:

- *Transparency* (advertence, consciousness): the user is told how attributes are collected, used and disclosed.
- *Controllability* (selection, consent): the user chooses what is released and agrees to it.
- *Minimization* (control, participation): only the information needed to access a resource is released.
- *Accountability* (traceability, policies): auditing and tracking actions, "in accordance with legal requirements", so that misuse and privacy breaches can be identified and audited.
- *Data quality* (validation, confidence): attributes are correct and validated.
- *Use limitation* (obligations, policies): only authorised entities use the data, for the stated reason; a policy can state "the reason or intention for using the data" and an obligation is a directive to be carried out before or after access.

TABLE II
COVERAGE OF THE NINE PRIVACY CHARACTERISTICS IN THE SURVEY BY WERNER ET AL. [11], COUNTED FROM ITS TABLES 1–3

Characteristic	Proposals (of 23, Table 2)	EU projects (of 6, Table 1)	Authors' works (of 4, Table 3)
Transparency	14	6	4
Controllability	17	6	3
Minimization	8	3	1
Accountability	**2**	3	**0**
Data quality	2	6	1
Use limitation	**5**	2	**0**
User-friendly	2	3	3 (all "partial")
Trust	5	2	2
Obfuscation	6	3	1

- *User-friendliness* (with or without add-ons): an interface lets users "check what data was released under what conditions, and to whom".
- *Trust* (reputation, risk): confidence in providers is assessed with metrics.
- *Obfuscation* (anonymity, pseudonym): users are not identified, or are identified only by pseudonyms.

The survey relates these characteristics to the three properties of Birrell and Schneider (confidentiality, undetectability, unlinkability) in a Venn diagram (its Fig. 4).

The survey then classifies six European research projects (PRIME, PrimeLife, ABC4Trust, GÉANT, PRISMACLOUD and CREDENTIAL; its Table 1) and 23 published proposals from 2009 to 2016 (its Table 2) by the characteristics they address. It describes five challenges for future research (its Section 4) and four works in progress by the authors' group (its Section 5 and Table 3). One of these works defines four standard *interaction profiles*, anonymous, pseudonym, partial attributes and total attributes, as OIDC scopes [35]; another adds a dissemination screen that warns the user about the provider's reputation [36].

B. Reading the tables

Table II counts how many of the surveyed proposals and projects address each characteristic. Two characteristics stand out. Accountability is marked for only 2 of the 23 proposals (Suriadi et al. [37] and Sánchez et al. [38]) and for 3 of the 6 projects. Use limitation is marked for 5 proposals and 2 projects. Data quality and user-friendliness are also rare, but they are less central to the problems described in Section I. In the authors' own works in progress, accountability and use limitation are not marked for any of the four.

The numbers suggest a pattern. Most work concentrates on the moment of disclosure: telling the user what will be released (transparency), letting the user choose (controllability) and releasing less (minimization). Far less work addresses what happens around and after the disclosure: whether the release can be traced and audited later (accountability), and whether the service's use is bound to a stated reason and time (use limitation). For students, these are exactly the missing pieces. A student who never sees a record of releases cannot exercise any control after the fact, and a promise of deletion that nobody records cannot be checked.

Two cautions apply to this reading. First, the counts are coarse. The survey marks a characteristic as present or absent; a mark says nothing about how deeply a work addresses it, and we did not re-score the 23 works ourselves. Second, accountability and use limitation are not the only rare characteristics: data quality and user-friendliness are also marked for only 2 of the 23 proposals. We do not choose accountability and use limitation because their counts are lowest. We choose them because they match the problem in Section I, where releases leave no record and nothing ties a release to a purpose or a deletion date, and because the survey itself names ensuring that policies are implemented as an open challenge (Section 4.5 of the survey). Data quality matters less in a campus federation, where the IdP is the institute's system of record, and we treat user-friendliness as future work that needs a user study (Section XI).

C. The survey's open challenges

The survey's Section 4 lists five challenges and directions for privacy-based IdM in the cloud. We summarise each and note what is still open.

Reliability assessment (Section 4.1 of the survey). Federation partners need a way to evaluate trust in each other, for example through provider reputation or risk metrics. The surveyed works that address trust mostly use reputation values or static trust levels, such as the fully trusted, semi-trusted and untrusted classes of Ferdous and Poet [2]. What is missing is a link from observed behaviour to the trust level: a provider that misbehaves keeps its level until someone changes it.

Agreement of privacy options (Section 4.2). The IdP and the service should agree in advance on the privacy features of their interaction, and the agreement should not be static. In the surveyed works, such agreements are policies or contracts outside the protocol. They are not bound to individual releases, and the service does not sign anything that could later be held against it.

Definition of interaction profiles (Section 4.3). Predefined profiles help users and services disclose the least information needed. The authors' own work defines the four profiles as OIDC scopes [35]. A profile scope says what the service *asks* for. On its own it does not stop a service from asking for a more revealing profile, and it does nothing about linkage across services through identifiers or session values.

Granular control over the dissemination of data (Section 4.4). Releasing individual attributes, not whole profiles, should be possible depending on context. This is largely solved by attribute filtering and consent screens in existing systems such as the Shibboleth extension of Orawiwattanakul et al. [39].

Ensuring the implementation of policies (Section 4.5). The survey states that "it is essential that the application of different privacy policies be guaranteed in an automated way" and calls compliance assessment "a key challenge" [11]. Sticky policies [40], [41] attach usage conditions to data, but the survey notes about one sticky-policy approach that it "does not make clear the means to ensure the effective implementation of sticky policies after receiving data".

D. Weaknesses of the surveyed approaches

Taken together with the coverage counts, the survey's own remarks point to four weaknesses in the state of the art it describes.

1) *No evidence trail.* Accountability is the least covered characteristic. Even where it is marked, the works audit the system or trace data inside a trusted environment; none gives the user, or an outside auditor, evidence of releases that the operator cannot later alter.
2) *Purposes without enforcement.* Use limitation appears as policies and obligations, but not as a promise that is checked on every release. A service's purpose and retention period are not recorded with the data that was released under them.
3) *Static trust.* Trust levels and reputation scores are inputs set by administrators or by user feedback; a provider that breaks its promises does not automatically get less data.
4) *Proposals without implementations.* The survey itself observes of several works that they do "not provide details and [do] not validate the proposal", offer "mathematical and logical evidences, without a practical implementation", or do "not make clear how to implement" a component [11]. Several of the works are also built on SAML federations, whereas current campus deployments increasingly use OIDC.

E. From gaps to requirements

Table III maps each challenge and weakness to a requirement for TrustTrail and to the mechanism that meets it. The mechanisms are described in Sections VI and VII, and Section IX-J compares TrustTrail with the authors' works on the same nine characteristics.

One requirement from the survey we deliberately do not take on. The survey lists data quality, the validation of attribute values, as a characteristic. In a campus federation the IdP is the institute's own system of record, so we treat its attribute values as correct and do not add validation.

IV. Related Work

Section III covered the works classified in the base survey. This section looks at work beyond the survey, grouped by the problem each line of work solves, and positions TrustTrail against it. Table IV summarises the comparison.

TABLE III
FROM THE SURVEY'S CHALLENGES TO TRUSTTRAIL'S REQUIREMENTS AND MECHANISMS

Challenge or weakness	Requirement for TrustTrail	Mechanism
Reliability assessment (4.1); static trust	Trust must follow observed behaviour and change what a service receives.	Trust score lowered by violations; tier cap enforced at Keycloak and at release (Sec. VII-D).
Agreement of privacy options (4.2); purposes without enforcement	The service must commit to attributes, purpose and deletion date before any release, in a form it cannot later deny.	Signed purpose declarations; releases refused outside them; signed receipts (Sec. VII-B).
Interaction profiles (4.3)	The IdP and the release service must both enforce the profile, and services must not be able to link students.	Keycloak scope assignment as tier cap; pairwise `sub`; `sid` removed by a custom mapper (Sec. VII-A).
Granular control (4.4)	A service may ask for a subset of its profile.	Per-request attribute list checked against declaration and profile (Sec. VII-B).
Ensuring policy implementation (4.5); no evidence trail	Every release and every rule break must be recorded so that later changes to anchored history are detectable, even when the operator makes them.	Hash-chained ledger, receipts, Merkle anchoring on Ethereum (Secs. VII-C, VII-E).
Proposals without implementations	The design must run on a current IdP and be measured.	Keycloak 26 implementation, 157 tests, evaluation (Secs. VIII, IX).

A. Privacy-preserving single sign-on

A standard OIDC provider learns every service that a user logs in to, which lets the IdP build a profile of the user's activity. Several designs remove this. SPRESSO by Fett, Küsters and Schmitz is a web SSO system in which the IdP does not learn which relying party the user logs in to [42]. Hammann, Sasse and Basin propose a privacy-preserving extension of OIDC with the same aim [43], and Kroschewski and Lehmann address how a relying party can be authenticated in OIDC without revealing it to the IdP [44]. UPPRESSO, so far available as a preprint, aims at SSO that is both untraceable by the IdP and unlinkable across relying parties [45]. On the security side, Fett et al. gave a formal analysis of OAuth 2.0 [46], Mainka et al. systematised attacks on OIDC [47], and Wang, Chen and Wang showed through traffic analysis that deployed SSO systems had serious logic flaws [48].

These works protect the user from the IdP, or protect the protocol from attackers. TrustTrail addresses a different question: what the relying parties receive, under which promise, and whether that can be checked later. The two concerns are complementary. In a campus federation the IdP is the institute itself and already holds the student's records, so IdP tracking is less of a concern than over-release to the many services around it. We list IdP tracking as a non-goal in Section V; a deployment that needs both could combine TrustTrail with one of these protocols.

B. Anonymous credentials and self-sovereign identity

Chaum proposed in 1985 that individuals use a different pseudonym with each organisation and prove facts about themselves with credentials that cannot be linked across organisations [49]. Camenisch and Lysyanskaya gave an efficient anonymous credential system [50], which IBM implemented as Idemix [51]; Microsoft's U-Prove offers similar selective disclosure [52]. The PRIME, PrimeLife and ABC4Trust projects in the base survey built on these systems. More recently, self-sovereign identity places credentials in a wallet controlled by the user [53], [54], and the W3C Verifiable Credentials data model standardises their format [55].

Credential systems give strong unlinkability and fine-grained disclosure, often stronger than pairwise identifiers, because even the issuer cannot link presentations. They also move work to the user: the student needs a wallet, and every service needs a verifier. More importantly for our goals, a credential presentation leaves no record that the user can later show to a third party, and nothing ties the verifier's use of the data to a declared purpose. TrustTrail takes the opposite trade-off. It keeps standard OIDC on the service side and gives up some unlinkability

(the institute's IdP and TrustTrail know the mapping between pseudonyms), in exchange for a central point where purpose, retention and the release record can be enforced and kept.

C. Policy-based control of disclosure

A large part of the base survey concerns policies that state how released data may be used. Sticky policies attach machine-readable usage conditions to data so that the conditions travel with it [40]. Chadwick and Fatema built a privacy-preserving authorisation system in which users' policies are stuck to their data and enforced by the cloud provider's policy decision points [41]. Orawiwattanakul et al. let users filter which attributes a Shibboleth IdP releases [39], and Suriadi, Foo and Jøsang designed a user-centric federated SSO system with privacy requirements on the attribute exchange [37]. Sánchez et al. combined privacy preferences with dynamic federation for consumer cloud services [38].

In these systems, the user or the IdP states the policy, and the service is trusted to honour it once it has the data, unless trusted hardware enforces it. TrustTrail inverts the direction of the commitment: the *service* states its purpose and retention, signs the statement with its own key, and TrustTrail checks every release against it. Usage control generalises these ideas. The UCON_{ABC} model of Park and Sandhu adds obligations and conditions that are checked before and during use, and can be re-evaluated while access continues [56]. Enforcing such obligations requires a reference monitor wherever the data is used. TrustTrail has a reference monitor only at the release point. Neither approach controls the data once it has been released. TrustTrail adds evidence: the service's signature makes the promise attributable, and the ledger records which data was released under which promise.

D. Consent and receipts

The Kantara Initiative's Consent Receipt specification proposes that a user receive a record, in a standard format, every time they give consent, listing the controller, the purposes and the data categories [34]. TrustTrail's release receipts follow the same idea but are produced per release, signed by the release service, and linked to a ledger entry and, through anchoring, to a public chain. Research on consent interfaces explains why the record matters. Utz et al. showed in a field study that the design of GDPR consent notices strongly changes what users accept [57], and Felt et al. found that few smartphone users pay attention to, or correctly understand, permission prompts [58]. Applications also tend to request more permissions than they use [7], and OAuth is often misused in mobile applications [8]. A consent screen alone is therefore a weak control. TrustTrail's design does not rely on the student reading a prompt: the profile cap and the declaration are checked whether or not the student pays attention, and the dashboard and receipts give a record that can be read later.

E. Tamper-evident logging

Secure audit logs have a long history. Schneier and Kelsey used hash chains and evolving keys so that log entries written before a compromise cannot be changed undetectably [13]. Ma and Tsudik proposed forward-secure sequential aggregate signatures for the same purpose, with compact verification [24]. Crosby and Wallach designed history trees, in which the logger commits to the whole log after each append and auditors can check both membership of an entry and consistency between two commitments efficiently [23]. Certificate Transparency applies Merkle-tree logs at Internet scale, with monitors and auditors that check that a log only grows [14], [26], [27].

These systems assume an outside party that receives and keeps commitments, such as an auditor, a monitor or a gossip protocol. TrustTrail has two such parties: students, who hold receipts that contain the hash of their own entries, and a public blockchain, which holds the Merkle root of each batch. It uses the RFC 6962 leaf and node hashing and leaves consistency proofs between successive trees for future work, because the contract already refuses any batch that does not continue exactly where the previous one ended.

A log's head can also be fixed without a blockchain. The operator can sign the head hash and publish it, for example on a web page or a mailing list; a trusted timestamping authority can sign it following RFC 3161 [59]; or independent witnesses can cosign it, as CT monitors do. Each option detects the same rewrites and truncations as anchoring, as long as someone outside the operator's control keeps the published value. They differ in whom the verifier must trust and in cost. Section X compares them and explains why we chose a public chain.

TABLE IV
TRUSTTRAIL COMPARED WITH RELATED APPROACHES (OUR READING)

Approach	Examples	What it protects	What it leaves open for our goals
Privacy profiles as OIDC scopes	[35], [36]	Less information released, chosen per interaction	Profile requested, not capped; no linkage protection; no record of releases
Privacy-preserving SSO	[42], [43], [45]	User's activity hidden from the IdP	What relying parties receive and do; no purpose binding or release record
Anonymous credentials, SSI	[51], [52], [55]	Strong unlinkability, even against the issuer; selective disclosure	Needs wallets and verifiers; nothing records the verifier's stated purpose or what it received for later audit
Sticky and user policies	[39]–[41]	Usage conditions attached to data	Enforcement after release; no evidence the service accepted the policy
Consent receipts	[34]	User gets a structured, possibly signed record of each consent	One receipt per consent, not per release; no shared log that makes omissions or edits visible
Usage control	[56]	Obligations checked before and during use	Needs a reference monitor wherever data is used
Tamper-evident logs	[13], [23], [26]	Integrity of a log against later edits	Needs external auditors; does not decide what is released
Blockchain access control	[60], [61], [63]	Verifiable permissions and logs	Records or rights on chain (erasure conflict); new client software
TrustTrail	this paper	Profile cap, pairwise IDs, signed purpose, tamper-evident release record, trust feedback	IdP sees all logins; use after release; operator controls the pseudonym map

F. Blockchains for privacy and accountability

Zyskind, Nathan and Pentland proposed using a blockchain as an access-control manager for personal data held off chain [60]. MedRec uses Ethereum smart contracts to record permissions over medical records stored by providers [61]. Neisse, Steri and Nai-Fovino studied blockchain-based data accountability and provenance tracking with the GDPR in mind [62], and Truong et al. proposed a blockchain-based solution for GDPR-compliant personal data management [63]. Timestamping services such as OpenTimestamps and Chainpoint anchor Merkle roots of arbitrary documents in public blockchains [15], [32].

Many of the blockchain-based designs keep access rights or usage logs themselves on chain. This makes the record verifiable but raises a conflict with data protection law: information written to a public chain cannot be erased, and even pseudonymous log records can be personal data. TrustTrail writes only Merkle roots and entry ranges to the chain. A root reveals nothing about the entries it covers, and the ledger with its pseudonyms stays under the institute's control. This places TrustTrail closer to the anchoring services than to the blockchain access-control designs, with the difference that the anchored data is an accountability ledger with an enforced contiguity rule.

G. Trust and reputation

Jøsang, Ismail and Boyd survey trust and reputation systems for online services [64]. Most of them aggregate ratings from many users. In the base survey, trust appears as provider reputation shown to the user [36] and as static classes of federation membership [2]. TrustTrail's trust score is simpler and more local: it is computed only from facts that TrustTrail observes directly (registered privacy policy, registration metadata and recorded violations), and its only effect is to lower the most revealing profile the service can obtain. The weights are starting values, which Section X discusses.

V. PROBLEM STATEMENT, THREAT MODEL AND DESIGN GOALS

A. Problem statement

A university runs an OIDC identity provider for its students and a set of services of different trustworthiness. We want each service to receive only the attributes it is entitled to and has declared a purpose for, to be unable to

link a student across services by identifier, and to be held to its declared purpose and retention period, as far as the institute can observe. Every release, and every observable attempt to break these rules, should leave a record that the student can inspect and that nobody, including the operator of the system, can later change without detection. A service that breaks the rules should automatically receive less in future. Services should be able to keep using standard OIDC.

B. Parties

- *Students* log in through the IdP and want to know what each service received.
- The *IdP* (Keycloak) authenticates students, holds their attribute records, and issues tokens.
- *TrustTrail* verifies declarations, releases attribute values, keeps the ledger, computes trust and anchors the ledger.
- *Services* are relying parties of differing trust: the library, the LMS and the examination portal in our deployment, and possibly third-party services.
- The *operator* administers TrustTrail and its database, and can reset a service's trust.
- A *verifier* is anyone who checks a receipt or the ledger against the chain: a student, an auditor or the institute's grievance officer.
- The *chain* is an Ethereum network on which the anchor contract is deployed.

C. Adversaries

We consider the following adversaries. Each is a role that a party may take, not necessarily a separate attacker.

A1 *Over-asking service.* A service requests attributes that its declaration does not cover, or a profile above the one its agreement allows.

A2 *Retaining service.* A service keeps using a declaration after its promised deletion date, for example by requesting the data again.

A3 *Colluding services.* Two or more services share what they received in order to link a student's activity across them, using identifiers, session values or authentication timestamps in the tokens.

A4 *Repudiating service.* A service later denies having promised a purpose or a deletion date, or denies having received data.

A5 *Insider with database access.* Someone with write access to TrustTrail's database file, possibly the operator, edits, deletes or reorders ledger entries, rewrites the whole chain from some entry onward, or cuts off the newest entries. We assume this adversary can remove the database's append-only triggers.

A6 *Forger.* An attacker presents a forged access token to obtain attributes, or forges a receipt to claim a release that did not happen.

D. Assumptions

- The IdP is honest and correctly configured: it authenticates students correctly, signs tokens with keys that only it holds, and grants a client only the scopes assigned to it.
- TrustTrail runs its code faithfully *at the time of a release*. Adversary A5 may tamper with its stored data afterwards, but we do not consider a TrustTrail that lies while writing an entry; Section X discusses this case.
- Each service keeps its declaration-signing key and its client secret private. A stolen key lets the thief sign in the service's name. Registering a key requires only the service's client secret, and the key registry is held by TrustTrail, so the client secret is the root of trust for non-repudiation.
- TrustTrail holds the receipt-signing key and the anchoring key, and owns the anchor contract. The operator can therefore anchor whatever its ledger contains; what it cannot do is change or remove a batch once it is on chain.
- The anchor contract runs on a chain that the operator cannot rewrite. A local development chain does not meet this assumption; a public network does.
- Communication uses TLS in a real deployment. Our local deployment uses plain HTTP between containers on one machine.
- Hash functions (SHA-256) and signature schemes (ECDSA P-256, RSA) are secure.

TABLE V
DESIGN GOALS

Goal	Statement	Adversaries	Survey characteristic
G1	A service receives at most the attributes of the profile allowed by its current tier, and only those its declaration covers.	A1	Minimization, controllability
G2	No identifier or session value in the tokens or releases lets services link a student across services. (Released attribute values may still link; see non-goals.)	A3	Obfuscation (pseudonym), unlinkability
G3	Every release is covered by a service-signed declaration of purpose and deletion date; requests outside the declaration or after the date are refused.	A1, A2	Use limitation
G4	A service cannot deny a declaration signed with its registered key, and TrustTrail cannot deny a release it receipted.	A4	Accountability (policies)
G5	Any change, deletion or truncation of entries that have been anchored on a chain the operator cannot rewrite is detectable, also when the operator makes it; newer entries are protected by the chain and by receipts.	A5	Accountability (traceability)
G6	Observed violations lower a service's trust and, through its tier, what it can obtain.	A1, A2	Trust
G7	A student can see every release and refused rule break that concerns them, and verify each one independently.	A4, A5	Transparency, user-friendliness
G8	Services use standard OIDC plus signed JSON; the added latency per login stays in the order of milliseconds.	–	(deployability)

E. Design goals

Table V lists the design goals, the adversaries each goal addresses, and the survey characteristics [11] each goal corresponds to.

F. Non-goals

Some threats are outside TrustTrail's scope. We list them explicitly so the guarantees are not over-read.

- *Use after release.* Once a service holds data, TrustTrail cannot see whether it is used for the declared purpose or deleted on time. Rule 2 catches only a service that asks again.
- *IdP tracking.* Keycloak sees every login. The protocols in Section IV address this.
- *Traffic analysis.* Services that compare login times or network addresses may still correlate pseudonyms. TrustTrail removes identifiers, not timing.
- *Linking through attribute values.* The attributes that a profile releases, such as department, programme and year, can together single out a student in a small cohort, and the total profile releases name, e-mail address and roll number. Colluding services that hold such values can join their records without any identifier. TrustTrail limits which attributes each service receives; it does not make the released values unlinkable.
- *A dishonest release service.* If TrustTrail itself omits a release from the ledger, anchoring cannot reveal an entry that was never written; only the receipt held by the student is counter-evidence.
- *Availability.* Denial-of-service against Keycloak, TrustTrail or the chain is not addressed.

VI. SYSTEM ARCHITECTURE

This section describes TrustTrail's components, the data each holds, and how a release flows through them. Section VII then describes each mechanism in detail.

A. Design principle: the IdP decides which, TrustTrail decides what

TrustTrail splits the disclosure decision between two components. Keycloak decides *which* profile a service may obtain: a service's Keycloak client is assigned only the profile scopes up to its tier cap, so a login that asks for more fails at the IdP. The tokens that Keycloak issues to services carry a pairwise `sub`, the granted scope and no personal claims. TrustTrail decides *what* that profile contains, and it is the only path by which a service obtains

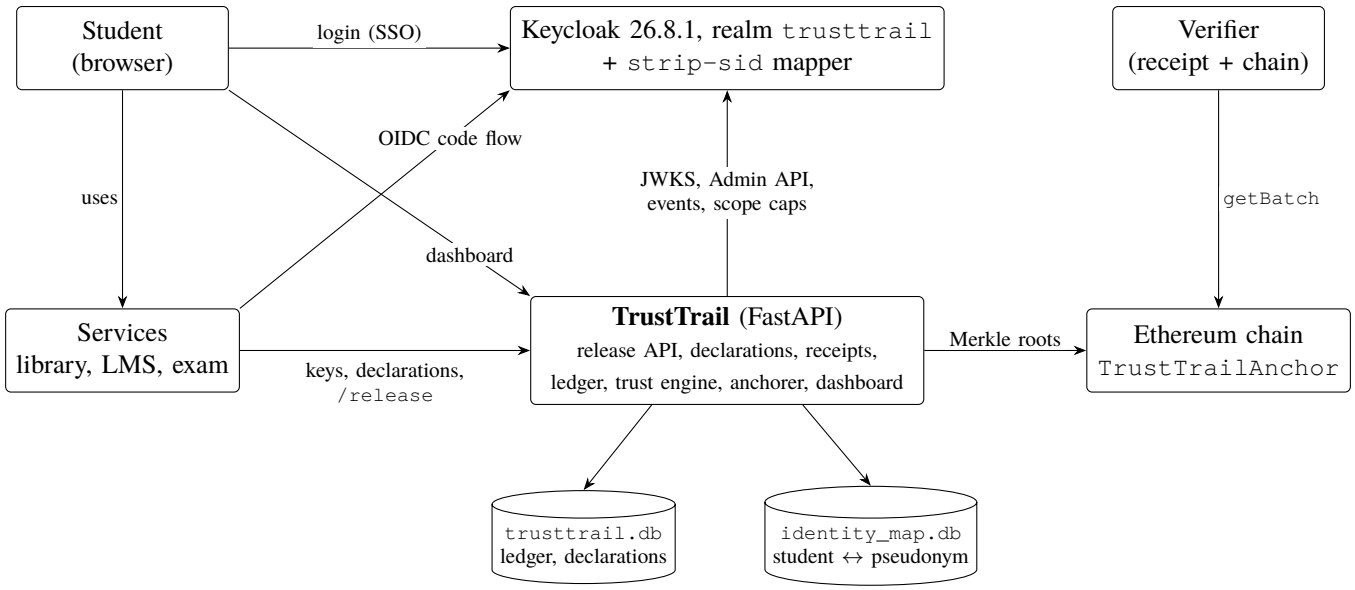


Fig. 2. TrustTrail components. Services speak standard OIDC to Keycloak and obtain attribute values only from TrustTrail. TrustTrail keeps the student-to-pseudonym map in a separate database from the ledger, and publishes only Merkle roots to the chain.

attribute values: the service presents the student's access token to TrustTrail's `/release` endpoint and receives the values that its profile and its declaration allow.

We chose this split for three reasons. First, it keeps the IdP standard. Services run an ordinary OIDC authorisation code flow, and the only change to Keycloak is one small protocol mapper and configuration through its Admin API. Second, it puts every release through a single point where the declaration can be checked and the ledger written. If attributes travelled in the ID token, the IdP would release them before any purpose check, and nobody would record the release. Third, it separates the two kinds of state. Keycloak keeps the student records it already had; TrustTrail keeps declarations, the ledger and trust state, and never stores attribute values.

B. Components

Figure 2 shows the components and their connections. All run as Docker containers defined in one Compose file.

1) Keycloak: Keycloak 26.8.1 is the IdP. A seed script creates the realm `trusttrail` through the Admin REST API. The realm defines four client scopes, `profile-anonymous`, `profile-pseudonym`, `profile-partial` and `profile-total`, which carry no claim mappers: their only effect is to appear in the token's `scope` claim. Each service client has every default scope removed except `basic`, and has as optional scopes only the profiles up to its tier cap. Each service client also has two protocol mappers: Keycloak's SHA-256 pairwise subject mapper and TrustTrail's custom `trusttrail-strip-sid-mapper`, a 72-line Java class that removes the session identifier from ID and access tokens. Keycloak records only `LOGIN_ERROR` events, which TrustTrail reads to detect over-cap login attempts.

2) TrustTrail: TrustTrail is a Python 3.12 application built with FastAPI. Its 20 modules (about 2,200 lines) implement:

- the *release API*, which verifies the student's access token, the declaration and the tier cap, and returns attribute values;
- *declarations*, which verifies and stores services' signed purpose declarations and their public keys;
- *receipts*, which signs a receipt for each release with TrustTrail's ES256 key;
- the *ledger*, an append-only, hash-chained SQLite table;
- the *trust engine*, which computes each service's score and tier and applies the tier's cap to the service's Keycloak client;
- the *anchorer*, which periodically builds a Merkle tree over new ledger entries and sends its root to the contract;
- the *dashboard*, where a student logs in and sees, verifies and downloads the record of every release concerning them.

Two background threads run inside TrustTrail: the trust thread synchronises with Keycloak every 3 s, and the anchoring thread runs every 300 s by default.

3) Services: Three invented campus services, a Digital Library, a Learning System and an Examination Portal, demonstrate honest behaviour. They share one small FastAPI application (136 lines) configured by a service identifier. Three further scripted services, which we call Campus Deals, Alumni Tracker and Career Portal, each break one rule on purpose and are used in the evaluation.

4) Chain: A local Hardhat 3 node runs an in-memory Ethereum chain, and a one-shot container deploys the `TrustTrailAnchor` contract (Solidity 0.8.28) and writes its address and ABI to a deployment file that TrustTrail reads. The same contract is deployed on the Sepolia public test network, and TrustTrail is switched between the two chains by configuration; Section IX-I reports measurements on both.

C. Data stores and trust boundaries

Each party holds a different part of the state, which limits what any single compromised store reveals.

- *Keycloak* holds the student records (name, e-mail, roll number, department, programme, year, affiliation), the client definitions and each client's random pairwise salt.
- *`trusttrail.db`* holds service public keys, declarations, the ledger, stored receipts, trust state, cached anchor metadata and processed event identifiers. The ledger contains attribute *names*, pseudonyms and hashes, never attribute values and never a student's real user ID. Because the identity map can link each pseudonym to a student, the ledger is still personal data in pseudonymised form (Section X-E).
- *`identity_map.db`* is a separate file that maps each (service, pseudonym) pair to the Keycloak user ID. It is needed to fetch attribute values and to show a student their own entries. Keeping it apart from the ledger means the ledger can be inspected, exported or audited without revealing who is who.
- TrustTrail's receipt-signing key, an EC P-256 private key, is generated on first use into TrustTrail's data volume with file mode 0600. Each service's declaration-signing key is generated into a volume that only the services mount. No private key is stored in the source repository.
- The *chain* holds, per batch, a 32-byte root, the first and last entry numbers and a timestamp.

D. End-to-end flow

Figure 3 shows the life of one release, from a service's setup to a student's verification. The steps are:

1) *Key registration.* The service obtains a client-credentials token from Keycloak and registers its public JWK at TrustTrail.
2) *Declaration.* The service signs a purpose declaration and submits it; TrustTrail verifies and stores it and returns its identifier.
3) *Login.* The student opens the service, which redirects to Keycloak asking for `openid profile-<x>`. Keycloak authenticates the student, or reuses the SSO session, and grants the profile only if it is assigned to the client. The service exchanges the code for tokens that carry the pairwise `sub`, no `sid` and no personal claims.
4) *Release.* The service calls `POST /release` with the student's access token and the declaration identifier. TrustTrail checks the token, the declaration, the deletion date and the tier cap, maps the pseudonym to the student, fetches the attribute values from Keycloak, and in one database transaction appends a ledger entry and signs a receipt. It returns the values and the receipt.
5) *Anchoring.* At the next anchoring interval, TrustTrail computes the Merkle root of all entries since the last batch and calls `anchor(root, first, last)` on the contract.
6) *Inspection.* The student logs in to the dashboard, sees the release, runs the verification checks and downloads a receipt bundle that can be verified later against the chain alone.

If any check in step 4 fails, TrustTrail refuses the release. If the refusal breaks one of the three rules, TrustTrail writes a violation entry and the trust engine may lower the service's tier, which removes profile scopes from its Keycloak client.

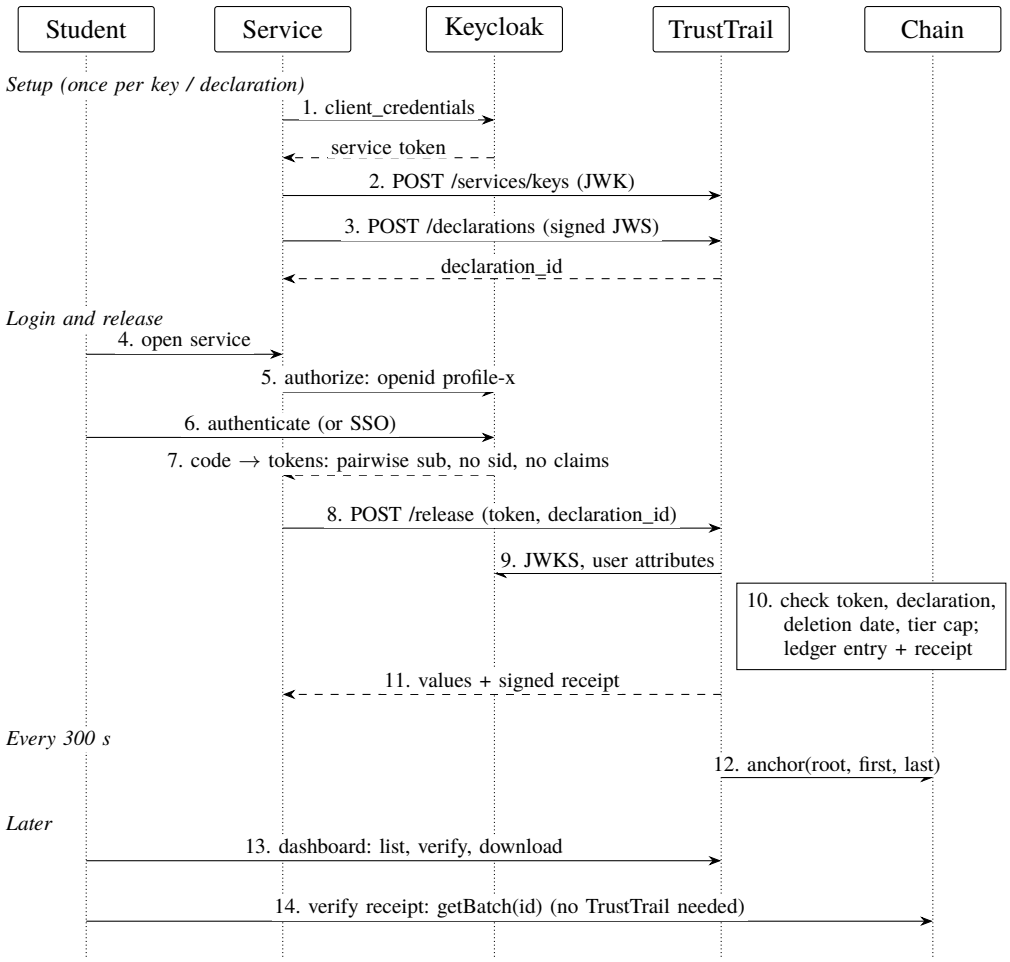


Fig. 3. Sequence of one release, from service setup to independent verification by the student.

TABLE VI
CONTAINERS IN THE DOCKER COMPOSE DEPLOYMENT

Container	Image / build	Role	Port
`keycloak`	Keycloak 26.8.1 + mapper JAR	IdP, realm `trusttrail`	8080
`chain`	Node 22, Hardhat 3	In-memory Ethereum chain	8545
`chain-deploy`	same as `chain`	Deploys `TrustTrailAnchor`, writes deployment file, exits	–
`trusttrail`	Python 3.12, FastAPI	Release API, ledger, trust, anchoring, dashboard	8000
`library`, `lms`, `exam`	Python 3.12, FastAPI	Honest demo services	8001–8003

E. Deployment

Table VI lists the containers. Keycloak's health check gates the start of TrustTrail and the services, and TrustTrail starts only after the contract has been deployed. TrustTrail's and the services' source code is mounted into their containers, so the tests and scripts run against the code in the repository.

VII. DETAILED DESIGN

This section describes TrustTrail's five mechanisms in the order in which they were built: disclosure profiles and pseudonyms (VII-A), purpose declarations and receipts (VII-B), the ledger and dashboard (VII-C), violations and trust (VII-D), and anchoring (VII-E). Section VII-F then brings them together in the release algorithm.

TABLE VII
DISCLOSURE PROFILES AND TIER CAPS

Profile	Released attributes	Cap of tier	Example service (base tier)
anonymous	affiliation	blocked	–
pseudonym	`sub`, affiliation	untrusted	Digital Library
partial	`sub`, affiliation, department, program, year	semi_trusted	Learning System
total	`sub`, affiliation, department, program, year, name, email, roll_no	fully_trusted	Examination Portal

A. Disclosure profiles, tier caps and pseudonyms

1) Profiles: TrustTrail adopts the four interaction profiles of Werner et al. [11], [35]. Table VII gives the attributes that each profile releases in our deployment. The profiles are nested: each one releases everything the previous one does, plus more. The attribute vocabulary has eight names. The name `sub` stands for the service's pairwise pseudonym; the other seven are values from the student's Keycloak record. Profiles are defined in a configuration file, so an institute can change their contents without changing code. We keep the survey's profile names, but the *anonymous* profile is not anonymous towards the service: TrustTrail releases no identifier under it, yet the service's Keycloak ID token still carries the service's pairwise `sub`, so the service can recognise a returning student. In the terms of Pfitzmann and Hansen [16] it gives relationship pseudonymity without attributes.

2) Tier caps at the IdP: Each service has a *base tier*, which is part of its agreement with the institute, and a *current tier*, which can only be equal to or lower than the base tier (Section VII-D). Each tier has a cap, the most revealing profile a service in that tier can obtain. TrustTrail enforces the current cap at Keycloak by assigning to the service's client exactly the optional scopes `profile-anonymous` up to `profile-`*cap*. A service that asks for a profile scope that is not assigned gets an `invalid_scope` error from Keycloak, and the student cannot log in to it with that request. The scopes have no claim mappers, so a granted profile only appears in the token's `scope` claim; it does not add any personal data to the token. To keep Keycloak from releasing personal data through its built-in scopes, the seed script removes every default scope from service clients except `basic`, and removes the `auth_time` mapper from `basic`.

3) Pairwise pseudonyms: Every service client has Keycloak's SHA-256 pairwise subject mapper with an empty configured salt, which makes Keycloak generate a random salt per client. For a student with Keycloak user ID u, service sector s (the host name of the service's redirect URIs) and client salt k, the pseudonym is

$$\mathit{sub}(u, s, k) = \text{UUIDv3}\big(\text{SHA-256}(s \parallel u \parallel k)\big), \tag{1}$$

where UUIDv3(x) is Java's `UUID.nameUUIDFromBytes`, an MD5-based version-3 UUID of x. Each demo service runs under its own host name (`library.localhost`, `lms.localhost`, `exam.localhost`), so each is its own sector.

TrustTrail needs to map a pseudonym back to the student in order to fetch attribute values. It recomputes every student's pseudonym for a service with a port of Keycloak's algorithm, using the salt read through the Admin API, and stores the pairs in `identity_map.db`. On an unknown pseudonym it rebuilds the table once. TrustTrail's service account can read the salts, so TrustTrail, and whoever controls it, can link every pseudonym to its student; this is by design, since TrustTrail must fetch the right student's attributes, and it is why the identity map is kept apart from the ledger. An integration test checks that the recomputed value equals the `sub` that Keycloak actually issues, for every student and every service.

4) Removing the shared session identifier: Pairwise identifiers alone did not give unlinkability in Keycloak 26. Keycloak adds the SSO session identifier `sid` to every token, and it has no built-in setting to turn this off. Every client that a student reaches within one SSO session receives the same `sid`. Two colluding services could therefore compare `sid` values and learn that pseudonym `f266e8cf…` at the library and pseudonym `4336de76…` at the LMS belong to the same student, which defeats the pairwise mapper.

We wrote a Keycloak protocol mapper, `trusttrail-strip-sid-mapper`, to remove it. It is a 72-line Java class that extends Keycloak's `AbstractOIDCProtocolMapper`, implements the access-token and ID-token mapper interfaces, and sets the token's session identifier to null in both. It is packaged as a JAR, built with Maven inside the Keycloak image's Docker build, and registered through Java's service loader. Two client settings remove the

remaining linkable values: `exclude.session.state.from.auth.response=true` drops `session_state` from the authorisation response, and removing the `auth_time` mapper drops the exact authentication time.

Removing `sid` has a cost. Service tokens no longer point to a Keycloak session, so refresh tokens (which we switched off), the userinfo endpoint and logout with `id_token_hint` do not work for the service clients. TrustTrail verifies access tokens locally and is not affected. Keycloak logs a warning at start-up that the protocol mapper interface is internal and may change between versions. A browser-level check script logs in to the library and then the LMS in one SSO session, using curl as the browser, and passes only if the two `sub` values differ and no `sid`, `session_state` or `auth_time` reaches either service. Deleting the mapper from the two clients makes the `sid` check fail, which shows that the mapper is what provides this property.

B. Purpose declarations and receipts

1) Access tokens: TrustTrail verifies every access token locally: the RS256 signature against Keycloak's published keys, the issuer, the expiry, and the audience. Each service client has a Keycloak audience mapper that adds `trusttrail` to the `aud` claim of its access tokens, and TrustTrail refuses any token whose audience does not include it. A valid token that Keycloak issued for some other resource server or client is therefore rejected, which follows the audience restriction recommended for OAuth resource servers [65], [66]. The audience value is the same for every service, so it does not help to link students. The calling service is identified by the token's `azp` claim. Revocation before expiry is not checked.

2) Service keys: Before declaring anything, a service registers its public key. It obtains a token with the OAuth client credentials grant, using its Keycloak client secret, and calls `POST /services/keys` with one or more public JWKs. TrustTrail accepts only tokens that carry a `client_id` claim equal to `azp`; Keycloak adds this claim only to client-credentials tokens, so a student's login token for the same service is rejected. This check relies on how Keycloak 26 builds its tokens. The JWT access-token profile of RFC 9068 [66] makes `client_id` required in all access tokens, so an IdP that follows it would need a different check, for example a dedicated audience for key registration; an integration test fails if a student's token is ever accepted. Keys must be EC or RSA, must have a `kid`, and must not contain private members. Our services use EC P-256 keys whose `kid` is the RFC 7638 thumbprint. The key registry is append-only: re-registering the same key is accepted, but reusing a `kid` for a different key returns `409 kid_conflict`. Old declarations therefore stay verifiable forever.

3) Declaration format and checks: A purpose declaration is a compact JWS. Listing 1 shows the header and payload of the LMS's declaration. The header must have `typ` equal to `purpose-declaration+jwt`, so that no other kind of JWT signed with the same key can be submitted as a declaration. The payload names the service, the audience `trusttrail`, the attributes, a free-text purpose of at most 500 characters, the deletion date `retention_until`, a fresh nonce and the validity period. A deletion date given as a calendar date means the end of that day in UTC; a timestamp with a time zone is taken exactly.

```
{ "alg": "ES256", "kid": "<RFC 7638 thumbprint of the LMS key>",
  "typ": "purpose-declaration+jwt" }
{ "client_id": "lms", "aud": "trusttrail",
  "attributes": ["sub", "affiliation", "department", "program", "year"],
  "purpose": "Enrol you in the courses of your department, program and year.",
  "retention_until": "2027-05-09",
  "nonce": "<uuid4>", "iat": 1794297600, "exp": 1796889600 }
```

Listing 1. Header and payload of a purpose declaration (LMS)

A service submits the JWS to `POST /declarations`. No bearer token is needed, because the signature authenticates the service. TrustTrail checks, in order: that the JWS parses and has the right `typ`; that `alg` is ES256 or RS256 (never `none` or an HMAC algorithm); that a registered key exists for the payload's `client_id` and the header's `kid`; the signature; the audience; that the declaration is not expired and not issued in the future (with 30 s of allowed clock skew) and is valid for at most 90 days; that the attributes are a non-empty list of known names; that the purpose is a non-empty string; and that the deletion date parses and is not already past. A nonce may be used only once per service, which stops an old declaration from being replayed as new. TrustTrail stores the declaration together with the original JWS and its SHA-256 hash, and identifies it by the first 16 hex digits of that hash. Table VIII lists the error codes.

TABLE VIII
REFUSALS RETURNED BY TRUSTTRAIL

Code	HTTP	Meaning	Violation
`malformed`, `unsupported_alg`, `unknown_key`	400	declaration cannot be parsed, uses a forbidden algorithm or an unregistered key	–
`bad_signature`, `wrong_audience`	400	declaration signature or audience is wrong	–
`expired`, `not_yet_valid`, `lifetime_too_long`	400	declaration outside its validity window or valid for over 90 days	–
`unknown_attribute`, `retention_in_past`	400	names outside the vocabulary; deletion date already passed	–
`replayed_nonce`, `kid_conflict`	409	nonce reused; `kid` reused for a different key	–
`missing_token`, `invalid_token`	401	no bearer token, or token not valid	–
`no_declaration`	403	declaration missing or belongs to another service	–
`declaration_expired`	403	declaration validity has ended	–
`retention_passed`	403	request after the promised deletion date	rule 2
`attribute_not_declared`	403	attribute not covered by the declaration	rule 1
`attribute_above_tier_cap`	403	attribute above the cap of the service's base tier	rule 3
`attribute_above_profile`	403	attribute above the current (possibly lowered) profile	–
`unknown_subject`	403	pseudonym cannot be mapped to a student	–

The key is looked up by the `client_id` in the unverified payload, and the signature check then proves that the holder of that service's key signed it. A service cannot submit a declaration in another service's name without the other service's private key.

4) Receipts: Every successful release produces a receipt: a JWS with `typ release-receipt+jwt`, signed with TrustTrail's own EC P-256 key. Its payload contains the issuer, a unique identifier, the service, the profile, the names (never the values) of the released attributes, the pseudonym (only if `sub` was released), the purpose, the deletion date, the declaration identifier and the SHA-256 hash of the declaration's JWS, the time of release, and the number and hash of the ledger entry. TrustTrail publishes its public key at `/.well-known/jwks.json`.

The two signatures give two kinds of evidence. The declaration, signed by the service, shows what the service promised, and the service cannot later deny it as long as its key was not stolen. The receipt, signed by TrustTrail, shows what TrustTrail released under that promise; its `declaration_sha256` binds the release to the exact declaration. A student who downloads a receipt holds evidence that does not depend on TrustTrail's database staying unchanged.

C. The hash-chained ledger and the student dashboard

1) Entry format: Every release and every rule-breaking refusal becomes one ledger entry. Since Phase 4 there is a third kind, `trust_reset`, written when an administrator forgives a service. Table IX lists the fields. Entries hold attribute names and the service's pseudonym for the student, never attribute values and never the student's Keycloak ID. Entry numbers are assigned by TrustTrail as $1, 2, 3, \ldots$, so a missing number is visible.

2) Hash chain: Let e_i be the content of entry i (all fields except `entry_hash`, including `prev_hash`) and $\mathrm{C}(\cdot)$ the canonical JSON encoding: keys sorted, no whitespace, UTF-8. Then

$$h_0 = 0^{256}, \qquad e_i.\textit{prev_hash} = h_{i-1}, \qquad h_i = \text{SHA-256}\big(\mathrm{C}(e_i)\big). \tag{2}$$

Because every value is a string, an integer or a sorted list of strings, this simple encoding is unambiguous without a full implementation of the JSON Canonicalization Scheme [67]. Verification walks the ledger from entry 1, recomputes each hash from the entry's content and stops at the first entry whose number, `prev_hash` or content does not match. SQLite triggers abort any `UPDATE` or `DELETE` on the ledger table. The triggers only stop changes made through normal use of the database; they are not a security boundary, which is why the hash chain, the receipts and the anchors exist.

TABLE IX
LEDGER ENTRY FIELDS

Field	Content
`entry_id`	1, 2, 3, …
`kind`	`release`, `violation` or `trust_reset`
`rule`	`undeclared_attribute`, `after_retention`, `above_tier_cap`, or empty
`time`	UTC timestamp with microseconds
`service_id`	the service's client ID
`student_pseudonym`	the pairwise `sub` at that service (empty for resets and login-time violations)
`profile`	profile granted or requested
`attributes_requested`	sorted names the service asked for
`attributes_released`	sorted names released (empty for a violation)
`purpose`, `retention_until`, `declaration_id`	copied from the declaration (for a reset, `purpose` holds the reason)
`service_signature`	the signature part of the service's declaration JWS
`prev_hash`	`entry_hash` of the previous entry; 64 zeros for entry 1
`entry_hash`	SHA-256 of the canonical JSON of all fields above

A release writes the ledger entry and stores the receipt in one database transaction. If either step fails, nothing is released. All database access goes through one lock, so concurrent releases cannot fork the chain; an integration test runs 10 releases concurrently and checks that the chain stays intact.

3) What the chain can and cannot detect: The chain on its own detects an entry whose content was changed and whose stored hash was left alone, an entry whose hash was recomputed (the next entry's `prev_hash` no longer matches), and a deleted or reordered entry. It cannot detect two attacks by an insider who controls the whole file: rewriting an entry and recomputing every later hash, and deleting the newest entries. Both leave a chain that is internally consistent. Detecting them needs a hash that is held outside the database. Receipts and anchors provide it, and the evaluation in Section IX-G tests all six cases.

4) Student dashboard: A student logs in to the dashboard through Keycloak with a separate client that does not use a pairwise mapper, so TrustTrail learns the student's real user ID. TrustTrail looks up the student's pseudonym at each service in `identity_map.db` and shows every ledger entry whose (service, pseudonym) pair matches, newest first: the service, the profile, the attributes released or requested, the purpose and the deletion date. Violations appear as highlighted rows. For each entry, a *Verify* button runs up to five checks: the entry matches its hash; the chain is intact up to this entry; the receipt is signed by TrustTrail; the receipt matches the entry; and, once the entry is anchored, its Merkle proof leads to the root stored on chain. A student can download a receipt bundle per entry, or all at once, containing the receipt, its decoded payload, TrustTrail's public key, the ledger entry, the service's original declaration JWS and the Merkle proof. A student can see only their own entries; requests for other entries return 404.

D. Violations, trust score and tier demotion

1) The three rules: TrustTrail can only judge behaviour that it observes. It therefore defines three violation rules, all of which concern requests:

1) *Undeclared attribute*: an explicit request names an attribute that the declaration does not cover. Detected in `/release`; refused with `attribute_not_declared`.
2) *After retention*: a release is requested under a declaration whose `retention_until` has passed. A service that asks again after the date by which it promised to delete the data is acting outside its promise. Detected in `/release`; refused with `retention_passed`.
3) *Above tier cap*: the service asks for more than its base tier allows. At release time this is an explicit attribute request above the base cap, refused with `attribute_above_tier_cap`. At login it is a request for a profile scope above the base cap, which Keycloak refuses with `invalid_scope`. TrustTrail learns of the login case

by reading Keycloak's `LOGIN_ERROR` events, whose reason reads "Invalid scopes: … profile-total". Each event is processed once and its identifier stored.

Every rule break is written to the ledger as a `violation` entry, with the rule, what was requested and nothing released, and is returned to the service in the refusal together with the entry number.

Two refusals are deliberately *not* violations. A request above the *current* profile but within the base tier's cap is refused as `attribute_above_profile` without a violation. Likewise, a demoted service that asks Keycloak for a profile that its agreement allows, but its current tier does not, is refused by Keycloak but not counted. Without this distinction, a single violation would demote the service, its next ordinary request would count as another violation, and one mistake would snowball into automatic blocking.

2) Score and tier: A service's score is computed from facts that TrustTrail can read from Keycloak and from its own ledger:

$$\mathit{score} = \max\Big(0,\ 100 - 15 \cdot [\text{no privacy policy}] - 10 \cdot [\text{metadata incomplete}] - 25 \cdot v\Big), \tag{3}$$

where $[\cdot]$ is 1 if the condition holds. "No privacy policy" means the Keycloak client has no `policyUri`; "metadata incomplete" means any of `name`, `description`, `rootUrl` or `tosUri` is missing; v is the number of violations in the last 90 days that came after the service's most recent trust reset. The tier is the base tier moved down one step for every full 25 points below 100:

$$d = \left\lfloor \frac{100 - \mathit{score}}{25} \right\rfloor, \qquad \mathit{tier} = \mathit{order}\big[\max(\text{index}(\mathit{base}) - d,\ 0)\big], \tag{4}$$

with $\mathit{order} = [\mathit{blocked}, \mathit{untrusted}, \mathit{semi_trusted}, \mathit{fully_trusted}]$. One violation therefore costs exactly one tier. A missing privacy policy alone (score 85) costs nothing, but together with incomplete metadata (score 75) it costs one tier. The configuration file states that every number is a starting value to tune, not a validated constant; we return to this in Section X.

3) Enforcement: After every violation, and every 3 s in a background thread, the trust engine recomputes each service's tier and makes Keycloak match it. It reads the service client's optional `profile-*` scopes through the Admin API, adds the missing ones and removes those above the cap. Because the background loop re-applies the cap continuously, a re-seeded or hand-edited Keycloak cannot silently undo a demotion. The cap is also applied a second time at release: a token issued before a demotion still carries the old profile in its `scope`, so `/release` lowers the profile to the current cap. An administrator can reset a service's trust, for example after the institute and the service have resolved a complaint; the reset is itself written to the ledger with its reason, and violations before it no longer count.

E. Merkle anchoring on Ethereum

1) Batches and Merkle roots: Every 300 s by default, the anchoring thread asks the contract for the last anchored entry number and reads up to 5,000 newer entries from the ledger. If there are none, it sends no transaction. Otherwise it builds a Merkle tree over their entry hashes following RFC 6962 [14]:

$$\mathit{leaf}_i = \text{SHA-256}(\texttt{0x00} \,\|\, h_i), \qquad \mathit{node} = \text{SHA-256}(\texttt{0x01} \,\|\, \mathit{left} \,\|\, \mathit{right}). \tag{5}$$

Adjacent nodes are paired level by level; an odd node at the end of a level is promoted unchanged. Promotion avoids a weakness of duplicating the last node, with which two different batches can share a root. The thread then calls `anchor(root, first, last)` and waits for the transaction receipt. All entry kinds are anchored, including violations and trust resets. Figure 4 illustrates a batch.

2) The anchor contract: Listing 2 shows the core of the `TrustTrailAnchor` contract (56 lines in total). It stores one record per batch: the root, the first and last entry numbers, and the block timestamp. Three rules make it a reliable reference. Only the owner, the account that deployed the contract and whose key TrustTrail holds, can anchor. A batch must cover at least one entry. And a batch must start exactly one entry after the previous batch ended, starting at entry 1. The last rule means the batches always cover the ledger from the first entry without gaps or overlaps, so anyone can see which entries are anchored, and a ledger that ends before the last anchored entry is visibly truncated. We deployed the contract both on a local Hardhat chain and on the Sepolia test network, at address `0x3415e1C6...74BC8`, where anyone can inspect its batches with a block explorer. The contract has no

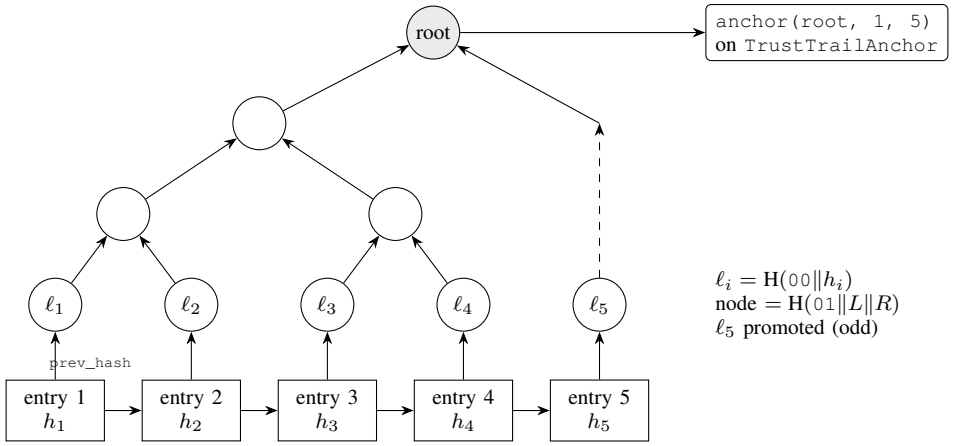


Fig. 4. A batch of five hash-chained ledger entries summarised as one Merkle root and anchored on chain. The proof for entry 3 is the sibling hashes ℓ_4, the node over ℓ_1, ℓ_2, and ℓ_5.

function that changes or deletes a batch; a contract test checks that `anchor` is the only function that writes state. Only hashes and numbers go on chain, never personal data.

```
struct Batch { bytes32 root; uint64 firstEntryId; uint64 lastEntryId; uint64 anchoredAt; }
address public immutable owner;
Batch[] private batches;
event Anchored(uint256 indexed batchId, bytes32 root,
           uint64 firstEntryId, uint64 lastEntryId, uint64 anchoredAt);

function anchor(bytes32 root, uint64 firstEntryId, uint64 lastEntryId)
   external returns (uint256 batchId) {
  if (msg.sender != owner) revert NotOwner();
  if (lastEntryId < firstEntryId) revert EmptyRange();
  uint64 expected = batches.length == 0 ? 1 : batches[batches.length - 1].lastEntryId + 1;
  if (firstEntryId != expected) revert NotContiguous(expected);
  batchId = batches.length;
  uint64 timestamp = uint64(block.timestamp);
  batches.push(Batch(root, firstEntryId, lastEntryId, timestamp));
  emit Anchored(batchId, root, firstEntryId, lastEntryId, timestamp);
}
```

Listing 2. Core of the anchor contract (Solidity 0.8.28)

3) Proofs and verification: `GET /ledger/proof/{id}` returns, for an anchored entry, its hash, the sibling hashes on the path to the root with their side, the batch, the contract address and the chain ID. It reveals no other entry's content. For an entry that is not yet anchored it returns `anchored: false` and the time until the next batch. `GET /anchors/verify` recomputes every anchored root from the ledger and compares it with the chain.

Verification always recomputes the leaf from the entry's *content*, not from the stored hash, so an edited entry whose stored hash was left unchanged also fails. A student can verify a downloaded receipt bundle without contacting TrustTrail, using a script that needs only the bundle file and a chain endpoint: it checks the receipt signature, checks that the ledger entry matches the receipt, recomputes the Merkle root from the proof, reads the batch from the contract and compares the roots and the entry range.

4) The anchoring window: An entry is protected by its anchor only after the next batch, which is up to one interval (300 s by default) plus the time to confirm the transaction. During this window, the entry is protected by the hash chain and by the receipt that its student may hold. The interval is a trade-off between cost (one transaction per interval when there is traffic) and the length of the window; Section IX-I measures both.

F. The release algorithm

Algorithm 1 brings the mechanisms together. The order of checks matters. The token and declaration are checked before anything else, so an unauthenticated caller cannot create ledger entries. The retention check comes before

Algorithm 1 `POST /release`

Require: bearer access token t; declaration ID δ; optional attribute list R

verify t with Keycloak's JWKS (RS256, issuer, audience `trusttrail`, type Bearer) ▷ else 401
$s \leftarrow t.azp$; $D \leftarrow$ declaration δ ▷ else 403 `no_declaration` if missing or $D.client \neq s$
$g \leftarrow$ highest `profile-*` in $t.scope$ (*anonymous* if none)
$p \leftarrow \min(g, \text{cap}(\text{tier}(s)))$ ▷ tokens issued before a demotion get today's cap
if $now > D.exp$ **then**
    return 403 `declaration_expired`
end if
if $now > D.retention_until$ **then**
    record violation (rule 2); **return** 403 `retention_passed`
end if
if $R = \perp$ **then**
    $A \leftarrow D.attributes \cap \text{attrs}(p)$
else
    if $R \not\subseteq D.attributes$ **then**
        record violation (rule 1); **return** 403 `attribute_not_declared`
    end if
    if $R \not\subseteq \text{attrs}(\text{cap}(\text{base}(s)))$ **then**
        record violation (rule 3); **return** 403 `attribute_above_tier_cap`
    end if
    if $R \not\subseteq \text{attrs}(p)$ **then**
        return 403 `attribute_above_profile` ▷ no violation
    end if
    $A \leftarrow R$
end if
$u \leftarrow$ student for $(s, t.sub)$ in identity map ▷ else 403 `unknown_subject`
$V \leftarrow$ values of A for u from Keycloak (`sub` $\mapsto t.sub$)
in one transaction: append ledger entry; sign receipt ρ; store ρ
return (p, V, ρ)

the attribute checks, so a service past its deletion date is told so whatever it asks for. When the service does not name attributes, TrustTrail releases the intersection of the declared attributes and the current profile, and this default request can never be a violation.

VIII. Implementation

We implemented TrustTrail in six phases. Each phase added one mechanism with its own tests and demonstration script: profiles and pseudonyms; declarations and receipts; the ledger and dashboard; violations and trust; anchoring; and the evaluation harness. This section describes the result. All students, services and secrets in the implementation are invented demo data.

A. Technology stack

Table X lists the main technologies. Keycloak suits the design because it is a widely used open-source IdP that supports pairwise identifiers, per-client scope assignment, an Admin REST API and plug-in protocol mappers. Python and FastAPI kept the TrustTrail service small. SQLite is enough for one release service at campus scale and keeps the ledger in a single file, which also made the tamper experiments easy to run on a copy. Hardhat provides a local Ethereum chain that starts in seconds, so the full system, including anchoring, runs on one laptop.

B. Code organisation and size

TrustTrail consists of 20 Python modules. The largest are the usability-study prototype (375 lines), the anchorer (179), the declaration verifier (170), the trust engine (168) and the dashboard (152). Table XI gives the size of each

TABLE X
TECHNOLOGY STACK

Layer	Technology	Use
Identity provider	Keycloak 26.8.1	OIDC, pairwise mapper, client scopes, Admin API, events
Keycloak extension	Java 21, Maven	`trusttrail-strip-sid-mapper` (protocol mapper SPI)
Release service	Python 3.12, FastAPI 0.162, Uvicorn	TrustTrail API, dashboard, background threads
JOSE	PyJWT 2.15 (with `cryptography`)	token verification, declarations, receipts
OIDC client	Authlib 1.9	dashboard login, demo services
Storage	SQLite (Python standard library)	ledger, declarations, trust state, identity map
Blockchain client	web3.py 7.16	anchoring, batch reads
Smart contract	Solidity 0.8.28 (optimiser, 200 runs)	`TrustTrailAnchor`
Chain and tests	Hardhat 3.20, ethers 6, Mocha, Node 22	local chain, deployment, contract tests
Deployment	Docker Compose	seven containers (Table VI)

TABLE XI
SIZE OF THE IMPLEMENTATION

Component	Files	Lines
TrustTrail application (Python)	20	2,159
Dashboard page (HTML, JavaScript)	1	139
Scripts: seeding, demonstrations, verifiers, bad services (Python)	9	1,025
Tests (Python)	14	1,198
Evaluation harness (Python, shell)	7	610
Demo service application (Python)	2	150
Anchor contract (Solidity)	1	56
Deployment script, contract tests, Hardhat configuration (TypeScript)	3	135
Keycloak protocol mapper (Java)	1	72
Browser-level check scripts (shell, Python)	3	224
Configuration (YAML)	7	255

part of the code base, counted with `wc -l` including comments and blank lines and excluding third-party code. In total the system is about 5,800 lines of code, of which about 1,200 are tests.

C. Configuration

Behaviour that an institute may want to change lives in seven YAML files rather than in code: the profiles and their attributes; the services with their URLs, base tiers and declarations; the invented students; the trust tiers, score weights and synchronisation interval; the anchoring interval, maximum batch size and transaction timeout; the declaration rules (accepted algorithms, maximum lifetime of 90 days, 30 s clock skew, 500-character purpose limit); and the usability-study scenarios. Secrets, such as client secrets, the administrator token and the anchoring key, are read from an environment file that is excluded from version control.

A seed script builds the Keycloak realm from these files through the Admin API: the four profile scopes, the service clients with their mappers and capped optional scopes, the dashboard client, the evaluation clients and the five students. The seed script deletes and recreates the realm, which makes demonstrations and tests repeatable. Re-seeding gives every client a new random pairwise salt, which changes every pseudonym and orphans existing ledger entries, so changes to a running realm are made by small idempotent scripts instead; for example, one script added the audience mapper of Section VII-B to the existing service clients.

D. API

Table XII summarises TrustTrail's HTTP API; Appendix A gives the full request and response formats. Endpoints that expose only hashes, scores and counts are public, so that anyone can check the ledger against the chain. Administrative endpoints require a bearer token compared in constant time.

TABLE XII
TRUSTTRAIL HTTP API (SUMMARY)

Endpoint	Caller	Purpose
`POST /services/keys`	service (client-credentials token)	register public signing keys
`POST /declarations`	service (signed JWS)	submit a purpose declaration
`GET /declarations/{id}`	public	read a stored declaration
`POST /release`	service (student's access token)	obtain attribute values and a receipt
`GET /.well-known/jwks.json`	public	TrustTrail's receipt-signing key
`GET /ledger/verify`	public	check the hash chain (no entry contents)
`GET /ledger/proof/{id}`	public	Merkle proof of one entry
`GET /anchors`, `GET /anchors/verify`	public	list batches; compare ledger with chain
`POST /anchors/run`	administrator	anchor now
`GET /trust`	public	score, tier and cap of every service
`POST /trust/sync`, `POST /trust/{s}/reset`	administrator	re-apply caps; forgive a service (recorded)
`/dashboard/...`	student (Keycloak login)	list, verify and download own entries

E. Demonstration services

The three honest services share one FastAPI application. On first use, a service generates its EC P-256 signing key into a Docker volume, registers the public key, and signs a declaration built from the configuration; it signs a fresh declaration when the current one is missing or expires within an hour. Its home page shows the declared purpose and, after login, the claims in its Keycloak ID token, the result of `/release` and the decoded receipt, which makes the separation between the two visible in a demonstration.

For the violation experiments, three scripted services each break exactly one rule. *Campus Deals* (semi-trusted) declares only `sub` and `affiliation` but asks for department and e-mail address as well (rule 1). *Alumni Tracker* (semi-trusted, and registered without a privacy policy, so its score starts at 85) declares a deletion time two seconds in the future, receives data once, waits three seconds and asks again (rule 2). *Career Portal* (untrusted) asks Keycloak for the `profile-total` scope (rule 3). Apart from their one misbehaviour they act normally: each asks for the lower of the profile it wants and its current public cap, so a demoted service adapts to its new tier.

F. Testing

The test suite has three layers.

- *94 unit tests* (70 test functions) run without the rest of the system: the pairwise algorithm, profile selection and the three rules, 16 cases of declaration verification (including `alg: none`, tampered payloads, another service's key and timestamps without a time zone), the ledger and its attacks, the trust formula, the Merkle tree with proofs for 1 to 33 leaves, ledger-against-chain verification, and the evaluation helpers.
- *57 integration tests* (43 functions) run against the seeded Docker stack, one file per phase. Among other things they check that service tokens name TrustTrail as audience and that a valid realm token issued for another client is refused, that every student has a different identifier at every service, that TrustTrail's recomputed pseudonyms match Keycloak's, that tokens carry no personal claims, that Keycloak enforces the tier cap, that a forged token is rejected, that 10 concurrent releases keep the chain intact, that each bad service is caught with the right rule and loses one tier per violation, that honest services get no violations, and that a downloaded receipt verifies against the chain and fails after its entry is edited.
- *6 contract tests* in Hardhat check that a batch is stored and the event emitted, that batches must be contiguous from entry 1, that empty ranges and non-owners are rejected, that `anchor` is the only state-changing function, and that anchoring a range of 100,000 entries does not cost more gas than the first anchor.

Three end-to-end scripts drive the real system as a user would: a browser-level SSO check (different `sub`, no `sid`, `session_state` or `auth_time`), a dashboard check (list, verify, download and independently verify a receipt;

another student's entry returns 404), and a scripted run of the usability-study screens. All 151 Python tests and the 6 contract tests passed on 10 November 2026, with TrustTrail anchoring on Sepolia.

G. Reproducibility

The whole system starts with two commands, `docker compose up -d --build` followed by the seed script, and the evaluation in Section IX runs with one more (`sh eval/run_all.sh`, about ten minutes). Appendix C lists the commands. The first build compiles the Keycloak mapper with Maven inside Docker and needs network access; later builds are cached.

IX. Evaluation

The evaluation answers the research questions of Section I with eight experiments: a functional check of what services receive (RQ1); the cost of the privacy layer in login latency, where that time goes, how the release service behaves under concurrent load, and how ledger and proof costs grow with size (RQ5); conformance tests for tampering with the ledger (RQ3) and for misbehaving services (RQ2, RQ4), and the cost and delay of anchoring on a local chain and on a public test network (RQ3, RQ5). It ends with a comparison against the base survey's taxonomy.

The tamper and violation experiments need a word of caution before the results. We wrote the attacks, the scripted services and the expected outcomes ourselves. These experiments are therefore *conformance tests*: they show that each mechanism behaves as designed against the attacks and misbehaviours we specified. They are not estimates of how often a real attacker or a real service would be caught.

A. Setup

All experiments ran on one Apple-silicon laptop under macOS, with every component in Docker containers as in Table VI: Keycloak 26.8.1 in development mode, TrustTrail, the three honest services and a local Hardhat chain. All data was invented: five students and six services. To exercise many logins quickly, the scripts obtain tokens with the OAuth password grant, which we enabled on the test clients for this purpose only. The latency and violation experiments were run on 10 November 2026 after the audience check of Section VII-B was added. For the public-chain experiments, the same day, we deployed the anchor contract on the Sepolia test network through an Alchemy RPC endpoint and switched TrustTrail's anchoring to it; the tamper experiment was run against both chains. Raw data is stored with the results.

B. What services receive

The Phase 1 demonstration script computes, for each of the five students, the `sub` that each of the three services receives, and asks TrustTrail for a release on behalf of each service. In every row, the three identifiers differed. For example, student *asha* is `f266e8cf-…` at the library, `4336de76-…` at the LMS and `158b3330-…` at the examination portal. The Keycloak tokens received by all three services contained no personal claims. TrustTrail released `{affiliation, sub}` to the library, added department, programme and year for the LMS, and added name, e-mail address and roll number for the examination portal, exactly the profiles of Table VII. The script also reads each service client's assigned scopes from Keycloak: the library can obtain only *anonymous* and *pseudonym*, the LMS also *partial*, and the examination portal all four. The browser-level check confirmed in a real authorisation code flow, with SSO between the library and the LMS, that the two services received different `sub` values and no `sid`, `session_state` or `auth_time`. This is a functional check with five students, not a measurement of unlinkability. It shows the absence of linking identifiers. It does not address linking through the released attribute values or through timing, both of which are non-goals (Section V).

C. Latency overhead

Method. We compare two ways of logging a student in to a service. The *baseline* is a plain Keycloak client that receives all attributes in its ID token, the usual configuration that TrustTrail replaces; one token request completes the login. The *TrustTrail* variant obtains the LMS's token (pairwise `sub`, no session identifier, no claims, audience

TABLE XIII
LOGIN LATENCY, 200 RUNS PER VARIANT (10 NOVEMBER 2026). PERCENTILES ARE NEAREST-RANK.

Variant	Median (ms)	p95 (ms)	Mean (ms)	Min (ms)	Max (ms)
Plain OIDC login (no privacy layer)	15.5	20.6	16.4	14.7	31.2
TrustTrail login + release (total)	18.6	24.0	19.5	17.4	41.2
of which: Keycloak token	15.3	18.8	15.9	14.5	37.5
of which: TrustTrail `/release`	3.5	4.9	3.6	2.7	6.4
Difference (TrustTrail − plain)	+3.2	+3.4			

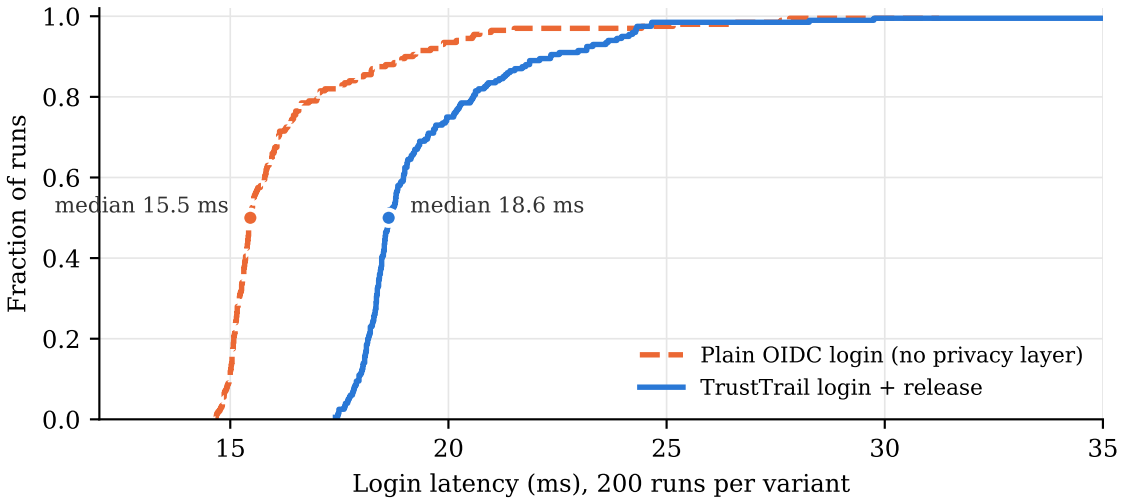


Fig. 5. Cumulative distribution of login latency. The TrustTrail curve is shifted right by the `/release` call, about 3 ms. One TrustTrail run at 41.2 ms lies beyond the plotted range.

`trusttrail`) and then calls `/release`, which verifies the token, applies the trust cap, checks the declaration, maps the pseudonym, fetches the attributes from Keycloak, appends a ledger entry and signs a receipt. After 10 warm-up runs of each, the harness ran 200 runs of each variant in a random interleaved order, cycling through the five students, and timed each with `time.perf_counter`. Percentiles use the nearest-rank definition. The confidence interval of the difference in medians is a percentile bootstrap with 10,000 resamples. Anchoring runs in the background and is not on the login path.

Results. Table XIII and Fig. 5 show the results. The median TrustTrail login took 18.6 ms against 15.5 ms for the baseline, a difference of 3.2 ms (95% CI 3.0 to 3.4 ms). The difference between the two 95th percentiles was 3.4 ms. The `/release` call itself had a median of 3.5 ms. Both variants had a standard deviation of about 2.5 ms. A run of the same benchmark before the audience check was added gave a median difference of 3.6 ms, which is within the variation we see between runs on this machine.

Interpretation. The absolute cost, about 3 ms per login, is the meaningful figure. The relative figure (about 21%) is large only because the baseline is a 15 ms token request over the loopback interface; a browser login with redirects and page loads, or any network between the parties, would make the same few milliseconds a much smaller fraction. A deployment would add one network round trip from the service to TrustTrail. This benchmark uses one service, one client at a time and the password grant; the next three subsections break the release time down by step, load the release service with concurrent clients, and grow the ledger.

D. Where the time inside a release goes

Method. A separate script runs the steps of `/release` (Algorithm 1) in the same order and with the same functions, inside one process, and times each step with `time.perf_counter`. It works on a copy of TrustTrail's

TABLE XIV
TIME PER STEP INSIDE /RELEASE, 200 RUNS (IN PROCESS, ON A COPY OF THE DATA)

Step	Median (ms)	p95 (ms)	Share of mean
Verify token (RS256, issuer, audience)	0.096	0.110	8%
Find declaration	0.016	0.022	1%
Apply trust cap	0.010	0.012	1%
Rule checks (declaration, retention, attributes)	0.005	0.007	0%
Map pseudonym to student	0.006	0.008	1%
Fetch attributes from Keycloak (HTTP)	0.503	0.657	43%
Build release	0.002	0.002	0%
Append ledger entry (hash chain)	0.047	0.070	4%
Sign receipt (ES256)	0.053	0.068	4%
Store receipt and commit	0.480	0.697	38%
Total in process	1.217	1.568	100%

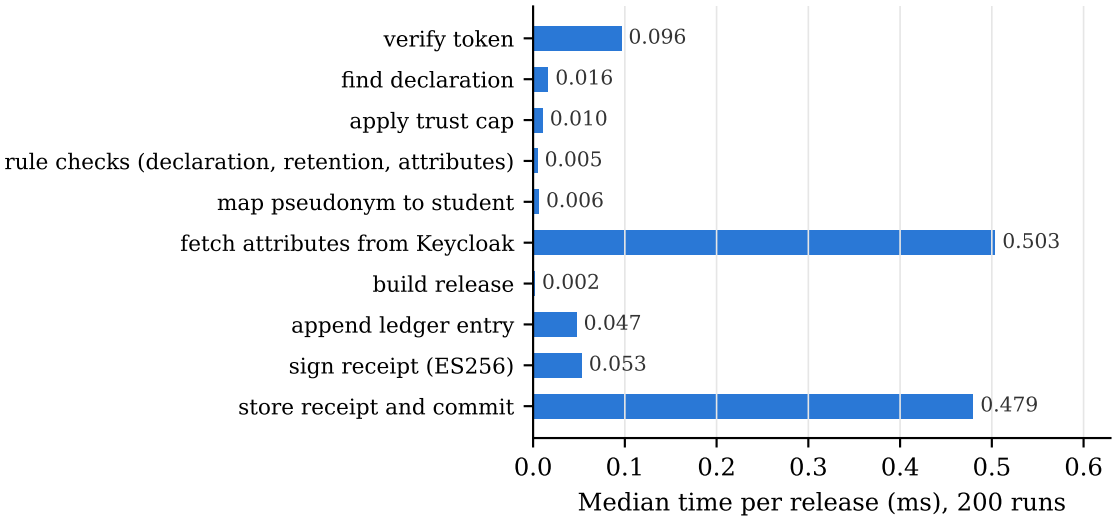


Fig. 6. Median time per step inside /release. The privacy checks themselves are negligible; the cost is one call to Keycloak and one durable database commit.

data directory, so the live ledger is not touched; the calls to Keycloak are real. We ran 200 releases for the LMS after 10 warm-up runs; tokens were obtained beforehand and are not timed.

Results. Table XIV and Fig. 6 show that two steps take most of the time. Fetching the student's record from Keycloak's Admin API took a median of 0.50 ms (43% of the mean), and storing the receipt and committing the SQLite transaction took 0.48 ms (38%). Verifying the token's RS256 signature took 0.10 ms, appending the ledger entry, including computing its SHA-256 hash, 0.05 ms, and signing the receipt with ES256 0.05 ms. All checks of the declaration, retention date, trust cap and attributes together took under 0.04 ms. The in-process total had a median of 1.2 ms, against a median of 3.5 ms for the same call over HTTP in Table XIII; the difference, about 2.3 ms, is spent in the HTTP server, FastAPI's request handling and JSON encoding.

Interpretation. The mechanisms that make TrustTrail privacy-preserving cost very little: the purpose, retention, trust and attribute checks together take microseconds, and the cryptography (hash chain, token verification, receipt signature) about 0.2 ms. The time goes to two pieces of plumbing. The call to Keycloak could be avoided by caching student records for a short time, at the cost of serving slightly stale attributes. The commit is the price of the rule "no release without a record": the ledger entry and receipt must be durable before the attributes leave TrustTrail. Most of the 3.5 ms that a service sees is HTTP handling, which a different web stack could reduce but which does not depend on the design.

TABLE XV
/RELEASE UNDER CONCURRENT LOAD (295 MEASURED CALLS PER LEVEL, ONE SERVER PROCESS)

Clients	Errors	Releases/s	Median (ms)	p95 (ms)	p99 (ms)	Max (ms)
1	0	212	2.0	2.6	3.2	4.1
10	0	468	18.0	27.9	34.5	41.8
25	0	534	41.1	66.4	82.1	93.3
50	0	489	90.3	139.2	184.6	209.3

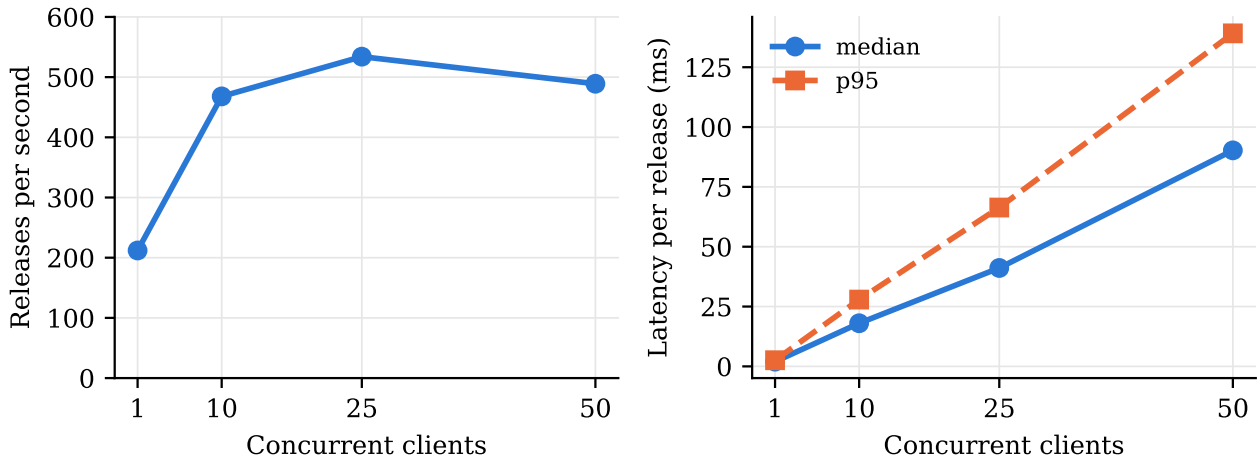


Fig. 7. Throughput (left) and latency (right) of `/release` as the number of concurrent clients grows. Throughput levels off at about 500 releases per second; beyond that point, extra clients only wait longer.

E. Behaviour under concurrent load

Method. The load script sends `/release` calls from 1, 10, 25 and 50 concurrent client threads. For each level it obtains 300 LMS tokens beforehand, sends five warm-up calls, then sends the remaining 295 calls as fast as the threads allow, and records each call's latency and the wall-clock time of the level. Every call is a real release that writes a ledger entry and a receipt. TrustTrail ran as one Uvicorn worker process, as in all other experiments. After the run we checked the hash chain.

Results. Table XV and Fig. 7 show that throughput rose from 212 releases per second with one client to a peak of 534 with 25 clients, and fell slightly to 489 with 50. Median latency grew roughly in proportion to the number of clients once throughput stopped rising: 2.0 ms with one client, 18 ms with 10, 41 ms with 25 and 90 ms with 50. All 1,180 measured calls succeeded, and the chain verified afterwards, with 2,649 entries.

Interpretation. One TrustTrail process saturates at about 500 releases per second on this laptop. Past that point the requests queue: latency grows with the number of waiting clients while throughput stays flat, which is the expected shape for a service whose writes pass through one lock and one Python process. The concurrency did not break the ledger: entry numbers and hash links stayed consistent under 50 simultaneous writers. For scale, a campus of 10,000 students in which every student logs in to five services within the same ten minutes produces about 83 releases per second, a sixth of the measured capacity. A larger deployment would run several processes in front of a database server that serialises only the ledger append. The single-client median here (2.0 ms) is lower than in Table XIII (3.5 ms) because this script sends only `/release` calls on a reused connection, while the latency benchmark alternates them with token requests to Keycloak.

F. How ledger and proof costs grow

Method. The scaling script builds synthetic ledgers of 1,000, 10,000 and 100,000 entries in a temporary SQLite file, using TrustTrail's own `ledger.append` with one transaction per entry, as in `/release`. For each size it

TABLE XVI
LEDGER AND MERKLE COSTS BY SIZE (SYNTHETIC ENTRIES, TEMPORARY SQLITE FILE)

Entries	Append (ms/entry)	Verify chain (s)	Storage (B/entry)	Merkle root (ms)	Proof (hashes)	Check proof (µs)
1,000	0.70	0.01	516	0.3	8–10	3.8
10,000	0.69	0.10	462	2.7	12–14	5.2
100,000	1.32	1.10	457	29.1	13–17	7.0

measures the append time per entry, the time to verify the whole chain from the first entry, the database size per entry, the time to compute a Merkle root over that many entries, and, for 200 random entries, the proof length and the time to check a proof.

Results. Table XVI shows that verification time grows linearly with the ledger: 0.01 s for 1,000 entries, 0.10 s for 10,000 and 1.1 s for 100,000. Storage was about 460 bytes per entry. Appending took about 0.7 ms per entry up to 10,000 entries and 1.3 ms at 100,000. Merkle proofs grow logarithmically: from at most 10 sibling hashes (320 bytes) at 1,000 entries to at most 17 (544 bytes) at 100,000, and checking one proof took under 10 µs at every size. Computing the root of 100,000 entries took 29 ms.

Interpretation. Full verification is linear but cheap: a campus ledger of 100,000 entries can be re-checked from the first entry in about a second, which is why the dashboard can afford to verify the whole chain on every visit. A student who checks a single receipt never needs the whole ledger: the proof is a few hundred bytes and takes microseconds to check, whatever the size of the batch. In TrustTrail a batch holds at most 5,000 entries, so real proofs have at most 13 hashes. Storage of about 0.5 KB per entry means that a million releases need about half a gigabyte, before receipts. The rise in append time at 100,000 entries, which we did not investigate further, still leaves appending far below the cost of a commit in Table XIV.

G. Tamper conformance

Method. The tamper experiment models adversary A5, an insider with write access to the database file. In each of five repetitions, the harness makes sure the ledger has at least six entries, anchors all of them, adds one more release after the anchor, and records the entry hashes contained in *all* stored receipts, which stand for receipts that students have downloaded. It then picks a random target release (not the first and not the newest) and runs six attacks, each on a fresh copy of the live database with the append-only triggers dropped:

A. change an entry's released attributes and leave its stored hash;
B. change an entry's purpose and recompute its hash;
C. delete the entry;
D. change the entry and recompute every later hash, so the chain is consistent again;
E. cut off the tail of the ledger, from the last anchored entry onward;
F. change the release made after the last anchor and fix the hashes from there on.

After each attack, three independent checks run: the hash chain verification, a check of every held receipt against the ledger (entry present and recomputed hash equal to the hash in the receipt), and verification of the ledger against the on-chain batches. The live ledger is never modified. We ran the experiment once with the local chain and once with Sepolia.

Results. Table XVII shows the outcome, which was identical on both chains: in all five repetitions, each check detected exactly the attacks the design says it should. The hash chain alone missed D, E and F. Anchoring caught every attack on anchored entries, including D and E, but not F, which edits an entry made after the last anchor. With Sepolia, the anchor check compares against roots held on a public network that TrustTrail cannot rewrite, so D and E, which include a full rewrite of history from some entry onward, are detected even if the operator makes them.

Interpretation. The receipts column depends on an assumption built into the harness: every affected entry has a receipt held outside the database. In practice only students who download their receipts hold one, so receipts protect the entries of those students and no others. Attack F is the case where this matters most. Until the next batch, an entry made after the last anchor is protected only if its student has downloaded the receipt; with the

TABLE XVII
TAMPER CONFORMANCE: REPETITIONS IN WHICH EACH CHECK DETECTED THE ATTACK (OF 5). IDENTICAL RESULTS WITH THE LOCAL CHAIN AND WITH SEPOLIA.

Attack	Hash chain	Student receipts*	Blockchain anchor	Any check
A. change an entry, leave its hash	5/5	5/5	5/5	5/5
B. change an entry, recompute its hash	5/5	5/5	5/5	5/5
C. delete an entry	5/5	5/5	5/5	5/5
D. change an entry, recompute every later hash	0/5	5/5	5/5	5/5
E. cut off the tail of the ledger	0/5	5/5	5/5	5/5
F. change an entry made after the last anchor	0/5	5/5	0/5	5/5

*Assumes the student whose entry was attacked holds its receipt; see text.

TABLE XVIII
VIOLATION CONFORMANCE: THREE REPETITIONS OF FIVE ATTEMPTS PER SCRIPTED SERVICE

Service	Misbehaviour (rule)	Recorded	Wrong rule	Tier after attempts 1–5
Campus Deals	asks for attributes its declaration does not cover (1)	15/15	0	semi_trusted → untrusted → blocked, stays blocked
Alumni Tracker	asks again after its deletion time (2)	15/15	0	semi_trusted → untrusted → blocked, stays blocked
Career Portal	asks Keycloak for a profile above its cap (3)	15/15	0	untrusted → blocked, stays blocked
Honest services	90 releases	0 false positives		no tier changes

default interval this window lasts at most about five minutes plus confirmation time. The other two mechanisms do not depend on student behaviour. The chain catches careless edits and locates them, and the anchors protect all anchored history for every student. Because the expected outcome of each cell was fixed before the run, the table confirms the design; the attacks that the design does not cover, such as TrustTrail omitting a release before it is ever written, are discussed in Section X.

H. Violation conformance and trust response

Method. The violation experiment models adversaries A1 and A2 with the three scripted services of Section VIII. Each of three repetitions starts by resetting every service to its base tier (the resets are recorded in the ledger). Honest traffic comes first: each honest service receives releases for all five students twice, 30 releases per repetition. Then each bad service misbehaves five times. After each attempt the harness triggers a trust synchronisation and waits until the violation count changes, at most 15 s, then records the score, tier and cap. An attempt counts as detected if a violation entry with the expected rule appears; a violation with another rule counts as a wrong rule; a violation recorded for an honest service is a false positive.

Results. Table XVIII shows that all 45 attempts produced a violation entry with the correct rule, and the 90 honest releases produced no violations and no tier changes. The first violation lowered each service by one tier. The semi-trusted services reached *blocked* at the second violation and the untrusted one at the first, after which they could obtain only the anonymous profile. Rule 3 at login, which TrustTrail learns about from Keycloak's event log, was recorded as reliably as the two rules checked inside `/release`.

Interpretation. The figure 15/15 overstates the amount of independent evidence. Each service starts from its base tier only three times, and after its second violation it is already blocked, so most of the 15 attempts test that a blocked service stays blocked and that further attempts are still recorded. What the experiment shows is that the three rules fire, that the feedback from violations to Keycloak's scope assignment works within the synchronisation interval of 3 s, and that ordinary traffic is not penalised. The Alumni Tracker reaches *blocked* after the same number of violations as Campus Deals although it starts at 85 points, because a score of 60 after one violation still means

TABLE XIX
GAS USED BY THE ANCHOR CONTRACT (10 NOVEMBER 2026)

Operation	Entries covered	Local Hardhat chain	Sepolia
Deploy `TrustTrailAnchor`	–	361,072	2,411,192
First anchor on a new contract	943 on Sepolia	91,659	349,631
Later anchors (measured batches)	6–7 local, 13–16 Sepolia	77,268 each	254,408 each
Later anchor (contract test)	100,000	77,268	–

TABLE XX
DELAY FROM LEDGER ENTRY TO CONFIRMED ANCHOR ON SEPOLIA (ANCHOR EVERY 30 S, A RELEASE ABOUT EVERY 2 S)

Batch	Entries	Min (s)	Median (s)	Max (s)	Transaction included after (s)
1	13	12.4	26.7	40.7	9.65
2	14	8.8	21.7	36.4	5.77
3	16	8.3	25.3	38.0	6.34
All	43		25.1	40.7	

one demotion. The rules see only requests: a service that misbehaves without asking TrustTrail for anything is invisible to them.

I. Anchoring cost and delay

Method. The anchoring script produces a release about every 2 s and triggers an anchor at a fixed interval for three batches. It records the gas used by each transaction, the time from sending a transaction to receiving its receipt, and the delay from each entry's timestamp to the confirmation of the transaction that anchors it. We ran it on the local chain with a 12 s interval and on Sepolia with a 30 s interval. On Sepolia we also anchored the existing ledger of 943 entries in one batch, and read every receipt back from the network to obtain gas and fees. The contract tests on the local chain measure an anchor that covers a range of 100,000 entries.

Results. Table XIX shows that gas per anchor did not depend on the number of entries. On the local chain, every anchor after the first cost 77,268 gas, for 6, 7 or 100,000 entries. On Sepolia, every anchor after the first cost 254,408 gas, for 13, 14 or 16 entries, and the first anchor, covering 943 entries, cost 349,631. The same contract used more gas on Sepolia than on the local chain for every operation; deployment took 2,411,192 gas against 361,072. We report the Sepolia figures as the realistic ones, since Hardhat's local chain does not necessarily apply the gas schedule of the current public network. Transactions on Sepolia were included in a block 5.8 to 9.7 s after they were sent, and the delay from entry to anchor had a median of 25.1 s and a maximum of 40.7 s over 43 entries with the 30 s interval (Table XX). On the local chain, which mines a block immediately, the delay was simply the timer: a median of 7.6 s and a maximum of 12.8 s with the 12 s interval. The whole Sepolia run, including deployment, cost 0.0024 SepoliaETH, almost all of it the deployment. Verifying the whole ledger against Sepolia took about 18 s for 15 batches, because it reads each batch from the network.

Interpretation. The cost of anchoring is constant per batch, because the contract stores one 32-byte root and three numbers whatever the batch size. On a network where gas has a price, the cost is 254,408 gas times that price per batch: at 1 gwei this is about 0.00025 ETH, and a day of continuous traffic at the default 300 s interval (288 batches) about 0.073 ETH. When nothing new is recorded, no transaction is sent. These cost figures are estimates for a priced network derived from the gas we measured; on Sepolia itself the gas price during our run was close to zero. The delay is dominated by the interval: with the default of 300 s, an entry waits on average about half the interval and at most one interval plus the time to include the transaction, which on Sepolia we measured at under 10 s. Our confirmation times are for inclusion in one block, not for finality, which on Ethereum takes longer; an auditor who wants a stronger guarantee should check anchors some minutes after they appear.

TABLE XXI

TRUSTTRAIL AND SIX SURVEYED WORKS ON THE NINE CHARACTERISTICS OF [11]. W1 = WEINGÄRTNER (SURVEY REF. [83]), W2 = WERNER ET AL. [35], W3 = BODNAR ET AL. (SURVEY REF. [24]), W4 = WERNER ET AL. [36] (SURVEY TABLE 3); S1 = SURIADI ET AL. [37], S2 = SÁNCHEZ ET AL. [38] (SURVEY TABLE 2). MARKS FOR W1–W4 AND S1–S2 ARE COPIED FROM THE SURVEY ("✓" = MARKED, "P" = PARTIAL). TRUSTTRAIL'S MARKS ARE OUR SELF-ASSESSMENT.

Characteristic	W1	W2	W3	W4	S1	S2	TrustTrail	Evidence for TrustTrail
Transparency	✓	✓	✓	✓	✓	✓	✓	declared purpose and deletion date; dashboard lists every release and violation
Controllability	✓		✓	✓	✓	✓	–	profiles set by tier, not chosen by the student; per-attribute choice only in a prototype
Minimization				✓	✓		✓	profile caps; tokens carry no claims; release = declared ∩ allowed
Accountability					✓	✓	✓	signed declarations and receipts; hash chain; anchoring on Sepolia; Table XVII
Data quality	✓				✓		–	IdP treated as the system of record
Use limitation					✓	✓	p	purpose and deletion date checked at every release; use after release not enforceable
User-friendly		p	p	p			p	dashboard and receipts; not yet evaluated with users
Trust	✓			✓		✓	p	behaviour-driven score and tier; heuristic weights
Obfuscation				✓		✓	p	pairwise `sub` and no `sid`; attribute values and the anonymous profile's `sub` can still link

J. Comparison with the base survey's taxonomy

Table XXI places TrustTrail in the taxonomy of Werner et al. [11]. For comparison it shows the four works in progress of the survey's Table 3 and the only two proposals in its Table 2 that are marked for accountability, by Suriadi et al. [37] and Sánchez et al. [38]. The marks for these six works are copied from the survey. The marks for TrustTrail are our own assessment, with the evidence from this paper; an assessment by authors of their own system is not independent, and we plan to have it repeated by someone who did not build TrustTrail. The survey's marks are binary or "partial", and so are ours; a mark records that a characteristic is addressed, not how well.

The comparison shows that TrustTrail is not the first work to address accountability and use limitation; Suriadi et al. and Sánchez et al. are marked for both. What the table cannot show is the form the accountability takes. The survey describes neither of those two works as producing evidence for the user, or an outside auditor, that the operator cannot later alter, which is the property that our tamper experiment tests. Against the authors' own four works, TrustTrail adds accountability, use limitation and behaviour-driven trust. It also gives something up. The authors' works let the user choose a profile at each interaction, which counts towards controllability. In TrustTrail the service's tier and declaration decide the profile, and the student does not choose per release in the running system. Our usability-study prototype adds per-attribute choice to a consent screen, but it is not part of the evaluated system (Section XI).

K. Answers to the research questions

- **RQ1** (profiles and linkage): yes, as far as identifiers are concerned, with standard OIDC on the service side. Each service received only its profile's attributes, a different pseudonym and no session identifier (Section IX-B); one custom Keycloak mapper was needed for the last property. Released attribute values can still link students and are outside what TrustTrail controls.
- **RQ2** (purpose binding): yes. No release happens without a declaration signed by the service, requests outside it or after its deletion date are refused and recorded, and each release has a receipt bound to the declaration's hash. Non-repudiation rests on the service's client secret, which authorises key registration.
- **RQ3** (tamper evidence): yes for anchored history. All six attacks were detected as designed, and on Sepolia the anchor check detects rewrites and truncation of anchored history even by the operator, with only hashes on chain (Table XVII). Entries newer than the last anchor rely on receipts that students may or may not hold, and a release that TrustTrail never records is not detected.

- **RQ4** (trust feedback): yes for the three observable rules; every scripted violation was recorded and demoted the service, with no false positives in 90 honest releases (Table XVIII).
- **RQ5** (cost): a median of 3.2 ms per login (95% CI 3.0–3.4 ms) on one machine, of which the privacy checks and cryptography are about 0.24 ms; about 500 releases per second from one server process; linear chain verification (1.1 s for 100,000 entries) with logarithmic proofs; and 254,408 gas per anchor on Sepolia regardless of batch size.

L. Threats to validity

Conclusion validity. The tamper and violation results are conformance tests whose expected outcomes we fixed in advance, so their 100% figures say that the implementation matches its design, not that the design catches a given share of real attacks. The violation experiment has three independent starts per service, and the tamper experiment five repetitions per attack. The receipt results assume that every affected student keeps receipts. *Internal validity.* The latency benchmark uses the password grant, which skips the browser redirects; both variants skip them equally, but the TrustTrail variant's token request also differs from the baseline's (pairwise identifier, audience and session mappers, no claim mappers). *External validity.* The latency, load, scaling and violation measurements come from one laptop with local containers, invented data, five students and six services; a deployment adds network latency between the parties, real attribute stores and more varied traffic. The load test used one server process and one service, and the scaling test synthetic entries. The anchoring cost and delay come from three batches on a test network with almost free gas; prices and inclusion times on a main network differ. *Construct validity.* The taxonomy marks for TrustTrail are a self-assessment, and the survey's characteristics are broad enough that reasonable assessors could disagree, especially about controllability and trust.

X. Discussion and Limitations

A. Evidence, not enforcement after release

TrustTrail's strongest guarantees concern what happens at TrustTrail: what a service promised, what it asked for, what it received, and whether the record of these facts has been changed. It has no view of what a service does with the data afterwards. A service that stays inside its declaration but sells the data, or never deletes it, receives a perfect trust score. Rule 2 catches only a service that comes back after its deletion date. This is a limit that TrustTrail shares with every approach that releases plaintext attributes, including sticky policies without trusted hardware. What TrustTrail adds is that the service's promise is signed and recorded with each release, so that when misuse is discovered by other means, for example through a complaint, an audit or a leak, the institute and the student can show exactly what the service promised and received. In the terms of the GDPR and the DPDP Act, TrustTrail supports the controller in *demonstrating* purpose limitation and accountability [9], [10]; it does not prove storage limitation.

B. Operator power and governance

TrustTrail concentrates power in whoever operates it, and anchoring limits only part of that power. The operator controls what enters the ledger: a dishonest TrustTrail could release data without writing an entry, and anchoring cannot reveal a record that was never written. The receipt is the only counter-evidence, and it exists only if the student, or the service, keeps it. An anchor proves that an entry is unchanged since the anchor, not that it was true when written. The operator holds the receipt-signing key, the anchoring key and the identity map that links pseudonyms to students, and it can reset any service's trust; resets are recorded in the ledger, but nothing prevents them. In a university, the operator is usually the institute itself, which is also the Data Fiduciary responsible for the services it oversees, so TrustTrail does not create an independent auditor by itself.

Governance measures can narrow this gap, and none of them needs a change to the design. TrustTrail's administration can sit with an office that does not run the services, such as the institute's data protection or grievance officer, with separate people holding the administrator token and the anchoring key. Every administrative action that changes the record (a trust reset) is already a ledger entry, so it is anchored like any release. The anchor contract's address can be published, as we do in this paper, so that students, auditors and the services themselves can check that batches keep arriving and that the ledger they are shown matches the chain. Services also have an interest in watching: a service that keeps its own receipts can show that a release it received was recorded, and a service accused of a violation can check the entry against the chain.

TABLE XXII
WAYS TO FIX THE HEAD OF THE LEDGER OUTSIDE THE OPERATOR'S CONTROL

Option	Verifier must trust	Weakness	Cost
Operator signs and publishes the head (web page, mailing list)	that someone archived the published value	the operator can replace the page; old values survive only if others kept them	none
Trusted timestamping authority (RFC 3161 [59])	the authority and its key	one more trusted party; tokens must be kept and the authority must stay available	low, often a paid service
Witnesses cosign the head (CT-style)	that enough witnesses are honest	needs independent organisations willing to run witnesses	operating cost for each witness
Public blockchain (TrustTrail)	the chain's consensus	fees; anchors are permanent and public; finality takes minutes	254,408 gas per batch

C. Linkability that remains

Pairwise identifiers and the removal of `sid`, `session_state` and `auth_time` remove *identifiers* that link a student across services. They do not remove timing. Two colluding services that compare the issue times of their tokens, or the network addresses of requests, could still guess which pseudonyms belong to the same student, especially in a small population. The anonymous profile is weaker than its name: TrustTrail releases no identifier under it, but the service's ID token from Keycloak still contains its pairwise `sub`, so the service can recognise a returning student. And Keycloak, as the IdP, sees every login. Privacy-preserving SSO protocols (Section IV) address IdP tracking and could be combined with TrustTrail.

The largest remaining channel is the released data itself. Department, programme and year together can single out a student in a small class, and the total profile releases name, e-mail address and roll number, which identify the student outright. Services that both hold such values can join their records without any identifier, so pairwise pseudonyms protect only students whose released attributes are common. The profile caps reduce this risk by giving low-trust services little to join on, but they do not remove it. Measuring it would need the size of the anonymity sets for each attribute combination in a real cohort, which we did not have.

D. Is a blockchain needed?

The anchor's job is to put the head of the ledger somewhere the operator cannot change it. A blockchain is one way to do this; Table XXII lists others. Each detects the same rewrites and truncations, provided the published value is kept by someone outside the operator's control. They differ in whom a verifier must trust, in cost, and in how much a verifier can check later.

We chose a public chain because, for a campus, it is the only option that needs no other organisation to cooperate. Anyone can check an anchor years later with nothing but a public RPC endpoint, as our receipt verifier does, and the contract itself enforces that batches are contiguous, so the operator cannot quietly skip or overlap ranges. The cost is a fee per batch and a dependence on the chain's continued operation. For an institute that already belongs to a federation with independent members, witness cosigning would be a reasonable alternative that avoids fees, and the ledger and receipts would not change.

E. The ledger as personal data

The ledger contains no attribute values, but it is still personal data. Each entry records that a pseudonymous student received a service at a given time, and the identity map, held by the same operator, can link the pseudonym to the student. Under the GDPR, pseudonymised data that can be attributed to a person with additional information remains personal data (Recital 26) [9], and the DPDP Act's duties attach to it too [10]. The current implementation keeps ledger entries indefinitely and offers no way to correct or erase them, which sits uneasily with the storage limitation that motivates the project and with a student's rights to correction and erasure (DPDP Act Section 12).

The append-only design does not have to conflict with erasure. Three measures fit it without weakening the tamper evidence. First, the institute can set a retention period for ledger entries, after which each expired entry is replaced

by a tombstone that keeps only its number and hash, and its identity-map row is deleted. The chain links and the Merkle roots can still be checked, because they depend only on the hashes; what is lost is the ability to recompute an expired entry's hash from its content, which is acceptable once the entry has served its purpose. Second, erasure on request can be implemented by deleting the student's rows from the identity map: the ledger entries then carry pseudonyms that nobody can attribute, while their hashes, and therefore the chain and the anchors, are unchanged. Third, correction is handled by appending a correcting entry, never by editing an old one. We have not implemented these measures, and whether deleting the mapping is enough to make the remaining entries anonymous is a legal question that depends on what else the institute holds.

The anchored data needs separate thought. A Merkle root is a hash of hashes of entries, each of which contains a pseudonym and a timestamp. It reveals nothing about the entries without the ledger, and once the ledger entries and their mapping are deleted, the root is not linkable to anyone. Whether such hashes count as personal data while the ledger still exists is not settled; the European Data Protection Board has issued draft guidelines on processing personal data through blockchain technologies [68]. TrustTrail's design keeps the on-chain data to the minimum that anchoring needs, a root and an entry range, and keeps everything else off chain where it can be deleted.

F. Deployment at an Indian institute

In DPDP Act terms, the institute running TrustTrail is the Data Fiduciary, students are Data Principals, and services that the institute engages to process student data are Data Processors; a third-party service that decides its own purposes would be a Data Fiduciary in its own right. TrustTrail's records map onto several duties of the Act. The declaration and the dashboard give the student the processing purposes and the list of services that received their data, the information a Data Principal may request under Section 11. The signed deletion date gives the institute a recorded point at which the specified purpose ends, which it needs in order to apply the erasure duty of Section 8(7) to its processors. And the ledger and receipts are the evidence that a grievance under Sections 8(10) and 13 would need.

TrustTrail does not provide the remedy itself. A student who sees a violation on the dashboard needs a way to act on it, and the natural path is the institute's grievance redressal mechanism, which the Act requires anyway. A deployment would link each dashboard entry to that mechanism, so that a complaint carries the receipt and the entry number. Two further points matter in practice. The lawful basis for each service's processing, consent or one of the Act's legitimate uses, is a decision for the institute, and TrustTrail's declarations record the purpose but not the basis. And some students may be under eighteen, for whom the Act requires verifiable parental consent before processing; TrustTrail would need to know a student's age, or be configured to give such students only the lowest profiles until consent is recorded.

G. Dependence on Keycloak internals

The `sid` problem shows how a standard feature can undo a privacy property. Keycloak 26 adds `sid` to every token with no setting to turn it off, so we wrote a mapper against an SPI that Keycloak marks as internal. Its removal of `sid` breaks refresh tokens, the userinfo endpoint and logout with `id_token_hint` for service clients. The client setting that hides `session_state` is deprecated in Keycloak 26.8; if a later version removes it, the value would return in the login redirect. Our browser-level check script fails in that case, so an upgrade that breaks the property would be noticed. Other IdPs would need equivalent configuration, and the same check, before TrustTrail could rely on them.

H. The trust model

The trust score is a heuristic. Its weights (15 points for a missing privacy policy, 10 for incomplete metadata, 25 per violation, one tier per 25 points) were chosen by hand and tested only with scripted clients. It ranks services by what TrustTrail can see, which is a narrow view of trustworthiness. Its strength is that it is simple and predictable: a service can see its own score through the public trust endpoint and knows that one violation costs one tier. Two design choices limit unfair effects. Requests within the service's agreement are never counted as violations after a demotion, so one mistake does not snowball. And violations expire after 90 days. Rule 3 at login depends on Keycloak's event log: if events are disabled or expire before TrustTrail reads them, the attempts are refused but not counted. TrustTrail's Keycloak service account holds the broad `manage-clients` role in order to change scope assignments; Keycloak's fine-grained admin permissions could narrow this.

I. Key custody

Non-repudiation rests on keys. A declaration binds a service only while its private key is not shared or stolen, and registering a key requires only the service's Keycloak client secret, so the client secret is the root of trust for key registration. TrustTrail's receipt key and the anchoring key are held by TrustTrail; whoever holds the anchoring key can add batches but cannot change or remove existing ones. TrustTrail checks declarations' attribute names, not the meaning of the purpose text: it cannot judge whether a stated purpose is honest or proportionate. That judgement belongs to the institute when it agrees a service's base tier.

J. Scale and deployment

The implementation targets a single campus. All database writes pass through one lock and one SQLite connection, and one server process reached about 500 releases per second before requests began to queue (Section IX-E); the pseudonym map is rebuilt by reading at most 1,000 users per request. These are implementation limits rather than design limits: the ledger needs only an append-only table with a serialised append, which a server database provides, and the identity map can be updated incrementally. Keycloak runs in development mode, the services use the password grant for test scripts, and the local chain uses Hardhat's public test keys, so its anchors prove nothing to outsiders. We ran the anchoring on Sepolia to show that the design works on a public network. Sepolia is a test network, which its operators could in principle reset, and its gas is nearly free; a production deployment would use a main network, where fees apply (Section IX-I). Verifying many batches over a public RPC endpoint also takes time, about 18 s for 15 batches in our run, so a verifier that checks often should cache batches it has already read.

K. Lessons

Three lessons from building TrustTrail may be useful to others. First, separating *which* profile from *what* it contains made the design simple: the IdP enforces the cap with a mechanism it already has, scope assignment, and the release service can be small. Second, every accountability mechanism we added closed a gap left by the previous one: the chain misses full rewrites, receipts cover only students who keep them, and anchors leave a window. None of them alone would have been enough, which matches the survey's observation that accountability is hard and under-served. Third, an automatic feedback loop needs care to be fair. Without the distinction between the base tier and the current tier, a single violation would demote a service, its next ordinary request would count as a new violation, and one mistake would end in blocking.

XI. Future Work

A. Usability study

TrustTrail's transparency and user-friendliness are claims about people, which our evaluation does not test. We have designed a usability study, and its materials and analysis scripts are implemented, but it has not yet been run. It compares two consent screens: Keycloak's default consent screen, which lists scopes such as "Email address" and "User profile" with Yes and No buttons, and a TrustTrail prototype screen that shows the service's declared purpose, its trust tier, the date of its signed promise, the deletion date, the attributes that are always shared, and a checkbox for each optional attribute with nothing pre-ticked.

The design is within-subject: each participant uses both screens in two scenarios, for four tasks in total, with the order of screens and scenarios counterbalanced across four groups. In one scenario (the Learning System, semi-trusted, data kept for 180 days) the intended choice is to share department, programme and year but not e-mail address or roll number; in the other (the Examination Portal, fully trusted, 365 days) the intended choice is to share everything. Participants act as an invented student, so no real personal data is involved. For each task the study records whether the participant made the intended choice, the decision time, and whether they could correctly say which attributes the service now holds; for each screen it asks what the purpose was, how long the data will be kept, and three five-point agreement items. Comprehension will be compared between the two screens with a paired exact sign test.

Two points limit what the study can show. The per-attribute choice exists only in the prototype screen, so the study tests an interface that the running system does not yet have. And the scenario that asks for a partial share is impossible on Keycloak's screen by design, so comprehension, not choice, is the fair comparison. We plan a study

with 15 to 20 students and will report it separately. Before it starts, we will submit the protocol to the institute's ethics committee, or obtain its written confirmation that approval is not needed. Because participants would be fellow students, recruitment needs care to avoid pressure: invitations will go out through a general notice rather than personal requests, participation will have no connection to any course or grade, and participants may stop at any time without giving a reason. Sessions use the invented persona only, store results under a random participant code with no names, and the data will be deleted after the report is graded; the consent form and data-handling plan in the repository will be completed with these details before recruitment.

B. Further directions

- *Student choice at release.* Bringing the prototype's per-attribute choice into the release flow would add controllability, the characteristic on which TrustTrail scores lowest against the survey's taxonomy. The declaration would then bound what a service may ask for, and the student would choose within that bound.
- *Main-network deployment and witnesses.* We anchored on the Sepolia test network. A production deployment would anchor on a main network or a low-cost layer-2 network, where fees and finality can be measured, or would use independent witnesses that cosign the head of the ledger (Section X).
- *Retention and erasure.* Implementing ledger retention with tombstones, erasure by deleting identity-map rows, and correction by appending entries, as sketched in Section X-E.
- *Consistency proofs.* Adding Certificate Transparency's consistency proofs [27] would let a student who holds an old root check that the ledger only grew, without downloading it.
- *Selective-disclosure credentials.* Issuing attributes as verifiable credentials with selective disclosure [55] could remove TrustTrail from the path of attribute values while keeping declarations and the ledger, and would hide the student's pseudonym map from the operator.
- *Federations with several IdPs.* Extending tiers and declarations across institutions, for example for shared library or examination services, raises the agreement questions that the survey describes for dynamic federations.
- *Independent scoring.* The taxonomy assessment in Table XXI should be repeated by assessors who did not build TrustTrail.
- *Scaling out and hardening.* Running several TrustTrail processes over a server database, caching student records to avoid the per-release Keycloak call (Section IX-D), load testing with realistic traffic, narrowing TrustTrail's Keycloak permissions, and running Keycloak in production mode.

XII. Conclusion

Single sign-on gave students one login for many services, and with it a default in which every service receives the same identifier and the same personal claims, with no stated purpose and no record. Starting from the survey of privacy strategies for identity management by Werner, Westphall and Westphall, we found that accountability and use limitation are the least covered of its nine characteristics, and that its challenge of ensuring that privacy policies are implemented remained open.

TrustTrail addresses these gaps for an OpenID Connect federation. Keycloak caps which disclosure profile each service can obtain and issues pairwise pseudonyms without a shared session identifier; TrustTrail releases attribute values only under a purpose and deletion date that the service has signed, records every release and every refused rule break in a hash-chained ledger, anchors the ledger's Merkle roots on Ethereum, and lowers the tier of services that break the rules. In our evaluation, the privacy layer added a median of 3.2 ms to a login on one machine; conformance tests showed that six tamper attacks and 45 scripted violations were detected as designed, with no false positives; and on the Sepolia test network each anchor after the first cost 254,408 gas regardless of batch size.

TrustTrail does not make a service delete data, and it does not hide logins from the identity provider. It makes the promises of services explicit, checks them at every release, and keeps a record that neither the services nor the operator can change without detection once it is anchored on a public chain. What it cannot show is a release that was never recorded, and the governance and data-protection measures of Section X matter as much as the cryptography.

Acknowledgment

The authors thank their project guide, Malay Kumar, Indian Institute of Information Technology Dharwad, for guidance throughout this work.

Use of AI Tools

The authors used Claude (Anthropic), an AI assistant, to help search for and verify related literature, extract technical facts from the project's source code, draft and edit the text of this report, and produce figures and tables. References were checked against DOI records, publisher pages or the issuing body's pages, and every number reported in the evaluation comes from the project's own measurement scripts. The authors reviewed all content and take full responsibility for it.

Data and Code Availability

All students, services, credentials and attribute values used in this work are invented; no real personal data was collected or processed. The source code, configuration, test suite, evaluation scripts and raw results (in the `eval/results` directory) are publicly available at https://github.com/desmond009/Trust-Trail. The anchor contract used in the evaluation is deployed on the Sepolia test network at address `0x3415e1C6B20E184EA493007A56589Cebf2074BC8`, and its batches can be inspected with any Sepolia block explorer.

Ethics Statement

This work involved no human participants. The usability study described in Section XI has not been conducted; before it is, the authors will submit its protocol to the institute's ethics committee, or obtain written confirmation that approval is not required, and will obtain informed consent from every participant.

Author Contributions

Vijender Yadav, Ansh Hardaha and Sourabh: **[to be completed by the authors]**.

Conflict of Interest

The authors declare no conflict of interest.

Funding

This work received no external funding.

Appendix A
API Reference

All refusals have the form `{"error": "<code>", "detail": "<text>"}`; refusals that break a rule at `/release` also carry `"violation": {"rule", "entry_id"}`. Table VIII lists the codes.

```
POST /services/keys Authorization: Bearer <client-credentials token>
  {"keys": [{"kty":"EC","crv":"P-256","x":"...","y":"...",
          "kid":"<RFC 7638 thumbprint>","alg":"ES256","use":"sig"}]}
  -> 200 {"client_id": "lms", "kids": ["..."]}

POST /declarations (no bearer token; the JWS signature authenticates)
  {"declaration": "<compact JWS, typ purpose-declaration+jwt>"}
  -> 201 {"declaration_id", "client_id", "nonce", "attributes", "purpose",
        "retention_until", "issued_at", "expires_at", "kid", "jws", "jws_sha256"}

POST /release Authorization: Bearer <student's access token>
  {"declaration_id": "226617abd0d39a21", "attributes": ["sub", "department"]}
  -> 200 {"service": "lms", "profile": "partial",
        "declaration_id": "226617abd0d39a21", "entry_id": 12,
        "released": {"sub": "97d4...", "department": "Computer Science"},
        "receipt": "<JWS, typ release-receipt+jwt>"}

GET /ledger/proof/{entry_id}
  -> {"entry_id", "anchored": true, "entry_hash",
     "proof": [{"hash", "side": "left"|"right"}, ...],
     "batch": {"batch_id", "first_entry_id", "last_entry_id", "root",
             "tx_hash", "block_number", "gas_used", "anchored_at", ...},
```

```
    "network", "chain_id", "contract"}
 -> {"entry_id", "anchored": false, "last_anchored_entry_id",
    "next_anchor_within_seconds"} (not yet anchored)

GET /anchors/verify
 -> {"ok", "batches_checked", "anchored_up_to", "unanchored_entries",
    "failures": [{"batch_id", "reason"}]}

GET /trust
 -> {"<service>": {"score", "privacy_policy_declared", "missing_metadata",
                "violations", "demotions", "base_tier", "tier", "cap",
                "violations_by_rule"}, ...}
```

Listing 3. Requests and responses of the main endpoints

The receipt payload contains `iss`, `iat`, `jti`, `service`, `profile`, `attributes` (names only), `subject` (only if the pseudonym was released), `purpose`, `retention_until`, `declaration_id`, `declaration_sha256`, `entry_id`, `entry_hash` and `released_at`. A receipt bundle downloaded from the dashboard contains the receipt JWS and its decoded payload, TrustTrail's public key set, the ledger entry, the service's declaration JWS and the Merkle proof.

Appendix B
Configuration Files

```
order: [anonymous, pseudonym, partial, total]
profiles:
  anonymous: {sub: false, attributes: [affiliation]}
  pseudonym: {sub: true, attributes: [affiliation]}
  partial: {sub: true, attributes: [affiliation, department, program, year]}
  total: {sub: true, attributes: [affiliation, department, program, year,
                                  name, email, roll_no]}
```

Listing 4. `config/profiles.yaml`

```
# Every number here is a starting value to tune, not a validated constant.
tiers:
  fully_trusted: total
  semi_trusted: partial
  untrusted: pseudonym
  blocked: anonymous
tier_order: [blocked, untrusted, semi_trusted, fully_trusted]
score:
  start: 100
  no_privacy_policy: 15
  incomplete_metadata: 10
  required_metadata: [name, description, rootUrl, tosUri]
  violation_step: 25
  tier_band: 25
  violation_window_days: 90
sync_seconds: 3
```

Listing 5. `config/trust.yaml` (comments shortened)

```
audience: trusttrail
algorithms: [ES256, RS256] # never "none" or HMAC
max_lifetime_days: 90
clock_skew_seconds: 30
max_purpose_length: 500

interval_seconds: 300 # ANCHOR_INTERVAL_SECONDS overrides
max_batch_entries: 5000
receipt_timeout_seconds: 180
```

Listing 6. `config/declarations.yaml` and `config/anchoring.yaml`

APPENDIX C
REPRODUCING THE RESULTS

```
git clone https://github.com/desmond009/Trust-Trail.git && cd Trust-Trail
cp .env.example .env # local demo values only
docker compose up -d --build # Keycloak, chain, TrustTrail, services
docker compose run --rm trusttrail python -m scripts.seed # (re)creates the realm

docker compose exec trusttrail pytest # 94 unit + 57 integration tests
docker compose run --rm chain-deploy npx hardhat test # 6 contract tests
scripts/check_browser_sso.sh # browser-level unlinkability check
scripts/check_dashboard.sh # dashboard and receipt check

docker compose exec trusttrail sh eval/run_all.sh # latency, tamper,
                                                  # violations, steps, load, scaling (~10 min)
docker compose exec trusttrail python -m scripts.demo_phase5 \
   --batches 3 --interval 12 # anchoring gas and delay
docker compose exec trusttrail python -m scripts.verify_anchors

docker compose exec trusttrail python -m eval.bench_release_steps --runs 200 # time per step
docker compose exec trusttrail python -m eval.bench_load --levels 1,10,25,50 # concurrency
docker compose exec trusttrail python -m eval.bench_scaling # ledger size
```

Listing 7. Commands to run the system, the tests and the evaluation

REFERENCES

[1] E. Birrell and F. B. Schneider, "Federated identity management systems: A privacy-based characterization," *IEEE Security & Privacy*, vol. 11, no. 5, pp. 36–48, Sep. 2013.

[2] M. S. Ferdous and R. Poet, "Managing dynamic identity federations using security assertion markup language," *J. Theor. Appl. Electron. Commer. Res.*, vol. 10, no. 2, pp. 53–76, May 2015.

[3] S. Cantor, J. Kemp, R. Philpott, and E. Maler, "Assertions and protocols for the OASIS security assertion markup language (SAML) V2.0," OASIS Standard. http://docs.oasis-open.org/security/saml/v2.0/saml-core-2.0-os.pdf, Mar. 2005, accessed: Nov. 10, 2026.

[4] N. Sakimura, J. Bradley, M. Jones, B. de Medeiros, and C. Mortimore, "OpenID Connect core 1.0 incorporating errata set 2," OpenID Foundation. https://openid.net/specs/openid-connect-core-1_0.html, 2023, accessed: Nov. 10, 2026.

[5] D. Hardt, "The OAuth 2.0 authorization framework," IETF, RFC 6749, Oct. 2012.

[6] *Keycloak Documentation*, https://www.keycloak.org/documentation, Keycloak / Cloud Native Computing Foundation, accessed: Nov. 10, 2026.

[7] A. P. Felt, E. Chin, S. Hanna, D. Song, and D. Wagner, "Android permissions demystified," in *Proc. 18th ACM Conf. Comput. Commun. Secur. (CCS)*, Oct. 2011, pp. 627–638.

[8] E. Y. Chen, Y. Pei, S. Chen, Y. Tian, R. Kotcher, and P. Tague, "OAuth demystified for mobile application developers," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Nov. 2014, pp. 892–903.

[9] European Parliament and Council of the European Union, "Regulation (EU) 2016/679 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation)," Official Journal of the European Union, L 119, pp. 1–88. https://eur-lex.europa.eu/eli/reg/2016/679/oj, Apr. 2016, accessed: Nov. 10, 2026.

[10] Government of India, "The digital personal data protection act, 2023 (no. 22 of 2023)," The Gazette of India, Extraordinary, Part II, Sec. 1, Aug. 11, 2023. https://www.meity.gov.in/static/uploads/2024/06/2bf1f0e9f04e6fb4f8fef35e82c42aa5.pdf, 2023, accessed: Nov. 10, 2026.

[11] J. Werner, C. M. Westphall, and C. B. Westphall, "Cloud identity management: A survey on privacy strategies," *Computer Networks*, vol. 122, pp. 29–42, Jul. 2017.

[12] M. Jones, J. Bradley, and N. Sakimura, "JSON web signature (JWS)," IETF, RFC 7515, May 2015.

[13] B. Schneier and J. Kelsey, "Secure audit logs to support computer forensics," *ACM Trans. Inf. Syst. Secur.*, vol. 2, no. 2, pp. 159–176, May 1999.

[14] B. Laurie, A. Langley, and E. Kasper, "Certificate transparency," IETF, RFC 6962, Jun. 2013.

[15] P. Todd, "OpenTimestamps: Scalable, trust-minimized, distributed timestamping with Bitcoin," https://petertodd.org/2016/opentimestamps-announcement, Sep. 2016, accessed: Nov. 10, 2026.

[16] A. Pfitzmann and M. Hansen, "A terminology for talking about privacy by data minimization: Anonymity, unlinkability, undetectability, unobservability, pseudonymity, and identity management," Version v0.34, TU Dresden and ULD Kiel. https://dud.inf.tu-dresden.de/literatur/Anon_Terminology_v0.34.pdf, Aug. 2010, accessed: Nov. 10, 2026.

[17] M. Jones, J. Bradley, and N. Sakimura, "JSON web token (JWT)," IETF, RFC 7519, May 2015.

[18] D. Temoshok, J. P. Richer, Y.-Y. Choong, J. Fenton, N. Lefkovitz, A. Regenscheid, and R. Galluzzo, "Digital identity guidelines: Federation and assertions," National Institute of Standards and Technology, Gaithersburg, MD, USA, NIST Special Publication 800-63C-4, Jul. 2025.

[19] M. Jones, "JSON web algorithms (JWA)," IETF, RFC 7518, May 2015.

[20] ——, "JSON web key (JWK)," IETF, RFC 7517, May 2015.

[21] M. Jones and N. Sakimura, "JSON web key (JWK) thumbprint," IETF, RFC 7638, Sep. 2015.

[22] S. Haber and W. S. Stornetta, "How to time-stamp a digital document," *J. Cryptol.*, vol. 3, no. 2, pp. 99–111, 1991.

[23] S. A. Crosby and D. S. Wallach, "Efficient data structures for tamper-evident logging," in *Proc. 18th USENIX Security Symp.*, 2009, accessed: Nov. 10, 2026.

[24] D. Ma and G. Tsudik, "Extended abstract: Forward-secure sequential aggregate authentication," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2007, pp. 86–91.

[25] R. C. Merkle, "A digital signature based on a conventional encryption function," in *Advances in Cryptology — CRYPTO '87*, ser. Lecture Notes in Computer Science, vol. 293. Springer, 1988, pp. 369–378.

[26] B. Laurie, "Certificate transparency," *Commun. ACM*, vol. 57, no. 10, pp. 40–46, Oct. 2014.

[27] B. Laurie, E. Messeri, and R. Stradling, "Certificate transparency version 2.0," IETF, RFC 9162, Dec. 2021.

[28] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," https://bitcoin.org/bitcoin.pdf, 2008, accessed: Nov. 10, 2026.

[29] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger (shanghai version)," Ethereum Yellow Paper. https://ethereum.github.io/yellowpaper/paper.pdf, 2025, accessed: Nov. 10, 2026.

[30] *Solidity Documentation*, https://docs.soliditylang.org/, Solidity Team, Ethereum Foundation, accessed: Nov. 10, 2026.

[31] *Hardhat Documentation*, https://hardhat.org/docs, Nomic Foundation, accessed: Nov. 10, 2026.

[32] Tierion, "Chainpoint: A blockchain proof and anchoring standard," https://chainpoint.org/, accessed: Nov. 10, 2026.

[33] A. Cavoukian, "Privacy by design: The 7 foundational principles — implementation and mapping of fair information practices," Information and Privacy Commissioner of Ontario (mirror). https://student.cs.uwaterloo.ca/~cs492/papers/7foundationalprinciples_longer.pdf, 2011, accessed: Nov. 10, 2026.

[34] M. Lizar and D. Turner, "Consent receipt specification, version 1.1.0," Kantara Initiative. https://kantara.atlassian.net/wiki/x/Noo1, Feb. 2018, accessed: Nov. 10, 2026.

[35] J. Werner, C. M. Westphall, G. Weingartner, G. A. Geronimo, and C. B. Westphall, "An approach to IdM with privacy in the cloud," in *Proc. IEEE Int. Conf. Comput. Inf. Technol.; Ubiquitous Comput. Commun.; Dependable, Autonomic Secure Comput.; Pervasive Intell. Comput. (CIT/IUCC/DASC/PICOM)*, Oct. 2015, pp. 168–175.

[36] J. Werner and C. M. Westphall, "A model for identity management with privacy in the cloud," in *Proc. IEEE Symp. Comput. Commun. (ISCC)*, Jun. 2016, pp. 463–468.

[37] S. Suriadi, E. Foo, and A. Jøsang, "A user-centric federated single sign-on system," *J. Netw. Comput. Appl.*, vol. 32, no. 2, pp. 388–401, Mar. 2009.

[38] R. Sanchez, F. Almenares, P. Arias, D. Diaz-Sanchez, and A. Marin, "Enhancing privacy and dynamic federation in IdM for consumer cloud computing," *IEEE Trans. Consum. Electron.*, vol. 58, no. 1, pp. 95–103, Feb. 2012.

[39] T. Orawiwattanakul, K. Yamaji, M. Nakamura, T. Kataoka, and N. Sonehara, "User-controlled privacy protection with attribute-filter mechanism for a federated SSO environment using Shibboleth," in *Proc. Int. Conf. P2P, Parallel, Grid, Cloud Internet Comput. (3PGCIC)*, Nov. 2010, pp. 243–249.

[40] S. Pearson and M. Casassa-Mont, "Sticky policies: An approach for managing privacy across multiple parties," *Computer*, vol. 44, no. 9, pp. 60–68, Sep. 2011.

[41] D. W. Chadwick and K. Fatema, "A privacy preserving authorisation system for the cloud," *J. Comput. Syst. Sci.*, vol. 78, no. 5, pp. 1359–1373, Sep. 2012.

[42] D. Fett, R. Küsters, and G. Schmitz, "SPRESSO: A secure, privacy-respecting single sign-on system for the web," in *Proc. 22nd ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Oct. 2015, pp. 1358–1369.

[43] S. Hammann, R. Sasse, and D. Basin, "Privacy-preserving OpenID Connect," in *Proc. 15th ACM Asia Conf. Comput. Commun. Secur. (ASIA CCS)*, Oct. 2020, pp. 277–289.

[44] M. Kroschewski and A. Lehmann, "Save the implicit flow? Enabling privacy-preserving RP authentication in OpenID Connect," in *Proc. Privacy Enhancing Technol. (PoPETs)*, vol. 2023, no. 4, Oct. 2023, pp. 96–116.

[45] C. Guo, J. Lin, Q. Cai, W. Wang, W. Zhu, J. Jing, Q. Wang, B. Zhao, and F. Li, "UPPRESSO: Untraceable and unlinkable privacy-preserving single sign-on services," arXiv:2110.10396 [cs.CR]. https://arxiv.org/abs/2110.10396, 2021, accessed: Nov. 10, 2026.

[46] D. Fett, R. Küsters, and G. Schmitz, "A comprehensive formal security analysis of OAuth 2.0," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Oct. 2016, pp. 1204–1215.

[47] C. Mainka, V. Mladenov, J. Schwenk, and T. Wich, "SoK: Single sign-on security — an evaluation of OpenID Connect," in *Proc. IEEE Eur. Symp. Secur. Privacy (EuroS&P)*, Apr. 2017, pp. 251–266.

[48] R. Wang, S. Chen, and X. Wang, "Signing me onto your accounts through Facebook and Google: A traffic-guided security study of commercially deployed single-sign-on web services," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2012, pp. 365–379.

[49] D. Chaum, "Security without identification: Transaction systems to make Big Brother obsolete," *Commun. ACM*, vol. 28, no. 10, pp. 1030–1044, Oct. 1985.

[50] J. Camenisch and A. Lysyanskaya, "An efficient system for non-transferable anonymous credentials with optional anonymity revocation," in *Advances in Cryptology — EUROCRYPT 2001*, ser. Lecture Notes in Computer Science, vol. 2045. Springer, 2001, pp. 93–118.

[51] J. Camenisch and E. V. Herreweghen, "Design and implementation of the idemix anonymous credential system," in *Proc. 9th ACM Conf. Comput. Commun. Secur. (CCS)*, Nov. 2002, pp. 21–30.

[52] C. Paquin and G. Zaverucha, "U-Prove cryptographic specification V1.1 (revision 3)," Microsoft Corporation, Tech. Rep., Dec. 2013, https://www.microsoft.com/en-us/research/project/u-prove/. Accessed: Nov. 10, 2026.

[53] C. Allen, "The path to self-sovereign identity," Life With Alacrity (blog). https://www.lifewithalacrity.com/2016/04/the-path-to-self-sovereign-identity.html, Apr. 2016, accessed: Nov. 10, 2026.

[54] A. Mühle, A. Grüner, T. Gayvoronskaya, and C. Meinel, "A survey on essential components of a self-sovereign identity," *Comput. Sci. Rev.*, vol. 30, pp. 80–86, Nov. 2018.

[55] W3C, "Verifiable credentials data model v2.0," W3C Recommendation. https://www.w3.org/TR/vc-data-model-2.0/, May 2025, accessed: Nov. 10, 2026.

[56] J. Park and R. Sandhu, "The UCON_{ABC} usage control model," *ACM Trans. Inf. Syst. Secur.*, vol. 7, no. 1, pp. 128–174, Feb. 2004.

[57] C. Utz, M. Degeling, S. Fahl, F. Schaub, and T. Holz, "(Un)informed consent: Studying GDPR consent notices in the field," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Nov. 2019, pp. 973–990.
[58] A. P. Felt, E. Ha, S. Egelman, A. Haney, E. Chin, and D. Wagner, "Android permissions: User attention, comprehension, and behavior," in *Proc. 8th Symp. Usable Privacy Secur. (SOUPS)*, Jul. 2012, pp. 1–14.
[59] C. Adams, P. Cain, D. Pinkas, and R. Zuccherato, "Internet X.509 public key infrastructure time-stamp protocol (TSP)," Internet Engineering Task Force, RFC 3161, Aug. 2001.
[60] G. Zyskind, O. Nathan, and A. Pentland, "Decentralizing privacy: Using blockchain to protect personal data," in *Proc. IEEE Secur. Privacy Workshops (SPW)*, May 2015, pp. 180–184.
[61] A. Azaria, A. Ekblaw, T. Vieira, and A. Lippman, "MedRec: Using blockchain for medical data access and permission management," in *Proc. 2nd Int. Conf. Open Big Data (OBD)*, Aug. 2016, pp. 25–30.
[62] R. Neisse, G. Steri, and I. Nai-Fovino, "A blockchain-based approach for data accountability and provenance tracking," in *Proc. 12th Int. Conf. Availability, Reliability Secur. (ARES)*, Aug. 2017, pp. 1–10.
[63] N. B. Truong, K. Sun, G. M. Lee, and Y. Guo, "GDPR-compliant personal data management: A blockchain-based solution," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 1746–1761, 2020.
[64] A. Jøsang, R. Ismail, and C. Boyd, "A survey of trust and reputation systems for online service provision," *Decis. Support Syst.*, vol. 43, no. 2, pp. 618–644, Mar. 2007.
[65] B. Campbell, J. Bradley, and H. Tschofenig, "Resource indicators for OAuth 2.0," Internet Engineering Task Force, RFC 8707, Feb. 2020.
[66] V. Bertocci, "JSON web token (JWT) profile for OAuth 2.0 access tokens," Internet Engineering Task Force, RFC 9068, Oct. 2021.
[67] A. Rundgren, B. Jordan, and S. Erdtman, "JSON canonicalization scheme (JCS)," IETF, RFC 8785, Jun. 2020.
[68] European Data Protection Board, "Guidelines 02/2025 on processing of personal data through blockchain technologies (version for public consultation)," https://www.edpb.europa.eu/our-work-tools/documents/public-consultations/2025/guidelines-022025-processing-personal-data_en, 2025, consultation closed 9 June 2025. Accessed: Nov. 10, 2026.